(Resize-able) Array

1. Introduction

Visualization of one of the simplest data structure in Computer Science: **Array** (and its sorted form) surprisingly has not been done in VisuAlgo since its inception 2011-January 2024...

Stay tuned while we improve this page and its features.

2. Motivation

(Compact) Array is among the easiest and the most versatile data structure in Computer Science. Array is built-in almost all programming languages, e.g., C++, Python ('array' is called as 'list' in Python), Java, etc.

We can use (Compact) Array to implement List ADT.

We can use (Compact) Array to solve many classic problems. When not being used as a List ADT implementation (where positional order matters), it is often beneficial to first sort the elements first so that we can utilize faster algorithms.

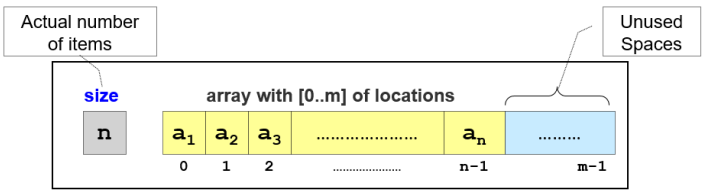
2-1. List ADT

Please see List ADT overview.

2-2. Array Implementation (Part 1)

(Compact) Array is a good candidate for implementing the List ADT as it is a simple construct to handle a collection of items.

When we say compact array, we mean an array that has **no gap**, i.e., if there are **N** items in the array (that has size **M**, where **M** ≥ **N**), then only index [0..**N**-1] are occupied and other indices [**N**..**M**-1] should remain **empty**.



Internal of the `list` ADT, Array Version

2-3. Array Implementation (Part 1)

Let the **compact** array name be `A` with index [0..**N**-1] occupied with the items of the list.

`get(i)`, just return `A[i]`.
This simple operation will be unnecessarily complicated if the array is **not** compact.

`search(v)`, we check each index `i` ∈ [0..**N**-1] one by one to see if `A[i] == v`.
This is because `v` (if it exists) can be anywhere in index [0..N-1].

Since this visualization only accept distinct items, `v` can only be found at most once.
In a general List ADT, we may want to have `search(v)` returns a list of indices.

`insert(i, v)`, we shift items ∈ [**i**..**N**-1] to [**i**+1..**N**] (*from backwards*) and set `A[i] = v`.
This is so that `v` is inserted correctly at index `i` and maintain compactness.

`remove(i)`, we shift items ∈ [**i+1**..**N**-1] to [**i**..**N**-2], overwriting the old `A[i]`.
This is to maintain compactness.

2-4. Time Complexity Summary

`get(i)` is very fast: Just one access, O(**1**).
Another CS course: 'Computer Organisation' discusses the details on this O(**1**) performance of this array indexing operation.

`search(v)`
In the best case, **v** is found at the first position, O(**1**).
In the worst case, **v** is not found in the list and we require O(**N**) scan to determine that.

`insert(i, v)`
In the best case, insert at **i = N**, there is no shifting of element, O(**1**).
In the worst case, insert at **i = 0**, we shift all **N** elements, O(**N**).

`remove(i)`
In the best case, remove at **i = N-1**, there is no shifting of element, O(**1**).
In the worst case, remove at **i = 0**, we shift all **N** elements, O(**N**).

2-5. Fixed Space Issue

The size of the compact array **M** is not infinite, but a finite number. This poses a problem as the maximum size may not be known in advance in many applications.

If **M** is too big, then the unused spaces are wasted.
If **M** is too small, then we will run out of space easily.

2-6. Variable Space

Solution: Make **M** a variable. So when the array is full, we create a larger array (usually two times larger) and move the elements from the old array to the new array. Thus, there is no more limits on size other than the (usually large) physical computer memory size.

C++ STL std::vector, Python list, Java Vector, or Java ArrayList all implement this variable-size array. Note that Python *list* and Java Array*List* are **not** Linked Lists, but are actually variable-size arrays. This array visualization implements this doubling-when-full strategy.

However, the classic array-based issues of space wastage and copying/shifting items overhead are still problematic.

2-7. Compact Array Applications

There are various applications that can be done on a Compact (Integer) Array **A**:

1. Searching for a specific value **v** in array **A**,
2. Finding the min/max or the k-th smallest/largest value in (static) array **A**,
3. Testing for uniqueness and deleting duplicates in array **A**,
4. Counting how many time a specific value **v** appear in array **A**,
5. Set intersection/union between array **A** and another sorted array **B**,
6. Finding a target pair **x** ∈ **A** and **y** ∈ **A** such that **x+y** equals to a target **z**,
7. Counting how many values in array **A** is inside range [**lo**..**hi**], etc.

See unsorted array and sorted array hints.

3. Actions

We will outline the possible actions that you can do in this page. For now, just try to guess based on the name of the function.

4. Visualizations

We will talk about the two modes: array (the content can be unsorted) versus sorted array (the content must always be sorted, without loss of generality: sorted in non-decreasing order).

5. (Unsorted) Array

There are already lots of (simple) applications that we can do with unsorted array.

5-1. Algorithm Ideas (Unsorted Array)

1. We can use O(**N**) linear search (leftmost to rightmost or vice versa) to find **v**,
2. For min/max, we can use O(**N**) linear search again;
   for k-th smallest/largest, we may need to use O(**kN**) algorithm[1],
3. We can use O(**N^2**) nested-loop to see if any two indices in **A** are the same,
4. We may need to use Hash Table to do this in O(**N**),
5. O(**N^2**) nested-loop is needed,
6. O(**N^2**) nested-loop is needed,
7. We may need to use Hash Table to do this in O(**N**).

There are better ways, especially if the array if sorted.

[1]There is a faster expected O(**N**) QuickSelect or O(**N**) worst-case linear time selection.

6. (Sorted) Array

When the array is sorted, we open up a lot of possibilities.

6-1. Algorithm Ideas (Sorted Array)

1. We can use O(log **N**) binary search on a sorted array,
2. A[0]/A[k-1]/A[N-k]/A[N-1] are the min/k-th smallest/k-th largest/max value in (static sorted) array **A**,
3. Duplicates, if any, will be next to each other in a sorted array **A**,
4. Same as above,
5. We can use modifications of merge routine of Merge Sort,
6. We can use two pointers method,
7. The index of **y** - the index of **x** + 1 (use two binary searches).

There can be other ways.

Sorting Problem and Sorting Algorithms

1. Introduction

Sorting is a very classic problem of reordering items (that can be compared, e.g., integers, floating-point numbers, strings, etc) of an array (or a list) in a certain order (increasing, non-decreasing (increasing or flat), decreasing, non-increasing (decreasing or flat), lexicographical, etc).

There are many different sorting algorithms, each has its own advantages and limitations.

Sorting is commonly used as the introductory problem in various Computer Science classes to showcase a range of algorithmic ideas.

Without loss of generality, we assume that we will sort only **Integers**, not necessarily distinct, in **non-decreasing order** in this visualization. Try clicking Bubble Sort for a sample animation of sorting the list of 5 jumbled integers (with duplicate) above.

1-1. Motivation - Interesting CS Ideas

Sorting problem has a variety of interesting algorithmic solutions that embody many Computer Science ideas:

1. Comparison versus non-comparison based strategies,
2. Iterative versus Recursive implementation,
3. Divide-and-Conquer paradigm (e.g., Merge Sort or Quick Sort),
4. Best/Worst/Average-case Time Complexity analysis,
5. Randomized Algorithms, etc.

1-2. Motivation - Applications

When an (integer) array **A** is sorted, many problems involving **A** become easy (or easier), please review the applications, the slower/harder unsorted array solutions, and the faster/easier sorted array solutions.

2. Actions

[This sentence is irrelevant for notes]

2-1. Define Your Own Input

The first action is about defining **your own** input, an array/a list **A** that is:

1. Totally random,
2. Random but sorted (in non-decreasing or non-increasing order),
3. Random but **nearly** sorted (in non-decreasing or non-increasing order),
4. Random and contain many duplicates (thus small range of integers), or
5. Defined solely by yourself.

[This sentence is irrelevant for notes]

2-2. Execute the Selected Sorting Algorithm

[This sentence is irrelevant for notes]

3. Visualisation

[This sentence is irrelevant for notes]

4. Common Sorting Algorithms

[This sentence is irrelevant for notes]

The first six algorithms in this module are **comparison-based** sorting algorithms while the last two are not. We will discuss this idea midway through this e-Lecture.

The middle three algorithms are **recursive** sorting algorithms while the rest are usually implemented iteratively.

4-1. Abbreviations

To save screen space, we abbreviate algorithm names into three characters each:

1. Comparison-based Sorting Algorithms:
   1. BUB - Bubble Sort,
   2. SEL - Selection Sort,
   3. INS - Insertion Sort,
   4. MER - Merge Sort (recursive implementation),
   5. QUI - Quick Sort (recursive implementation),
   6. R-Q - Random Quick Sort (recursive implementation).
2. Not Comparison-based Sorting Algorithms:
   1. COU - Counting Sort,
   2. RAD - Radix Sort.

5. 3 O(N^2) Comparison-based Sorting Algorithms

We will discuss three comparison-based sorting algorithms in the next few slides:

1. Bubble Sort,
2. Selection Sort,
3. Insertion Sort.

They are called **comparison-based** as they compare pairs of elements of the array and decide whether to swap them or not.

These three sorting algorithms are the easiest to implement but also not the most efficient, as they run in O(N^2).

6. Analysis of Algorithms (Basics)

Before we start with the discussion of various sorting algorithms, it may be a good idea to discuss the basics of asymptotic algorithm analysis, so that you can follow the discussions of the various O(**N**^2), O(**N** log **N**), and special O(**N**) sorting algorithms later.

This section can be skipped if you already know this topic.

6-1. Mathematical Pre-requisites

You need to already understand/remember all these:
-. Logarithm and Exponentiation, e.g., $\log_2(1024) = 10$, $2^{10} = 1024$
-. Arithmetic progression, e.g., 1+2+3+4+…+10 = 10*11/2 = 55
-. Geometric progression, e.g., 1+2+4+8+..+1024 = 1*(1-2^{11})/(1-2) = 2047
-. Linear/Quadratic/Cubic function, e.g., f1(x) = x+2, f2(x) = x^2+x-1, f3(x) = x^3+2x^2-x+7
-. Ceiling, Floor, and Absolute function, e.g., ceil(3.1) = 4, floor(3.1) = 3, abs(-7) = 7

6-2. What Is It?

Analysis of Algorithm is a process to evaluate rigorously the resources (time and space) needed by an algorithm and represent the result of the evaluation with a (simple) formula.

The time/space requirement of an algorithm is also called the time/space complexity of the algorithm, respectively.

For this module, we focus more on time requirement of various sorting algorithms.

6-3. Measuring the Actual Running Time?

We can measure the actual running time of a program by using wall clock time or by inserting timing-measurement code into our program, e.g., see the code shown in SpeedTest.cpp | py | java.

However, actual running time is not meaningful when comparing two algorithms as they are possibly coded in different languages, using different data sets, or running on different computers.

6-4. Counting # of Operations (1)

Instead of measuring the actual timing, we count the # of operations (arithmetic, assignment, comparison, etc). This is a way to assess its efficiency as an algorithm's execution time is correlated to the # of operations that it requires.

See the code shown in SpeedTest.cpp | py | java and the comments (especially on how to get the final value of variable counter).

Knowing the (precise) number of operations required by the algorithm, we can state something like this: Algorithm **X** takes **$2n^2$ + 100n** operations to solve problem of size **n**.

6-5. Counting # of Operations (2)

If the time **t** needed for one operation is known, then we can state that algorithm **X** takes **($2n^2$ + 100n)t** time units to solve problem of size **n**.

However, time **t** is dependent on the factors mentioned earlier, e.g., different languages, compilers and computers, the complexity of the operation itself (addition/subtraction is easier/faster to compute than multiplication/division), etc.

Therefore, instead of tying the analysis to actual time **t**, we can state that algorithm **X** takes time that is **proportional to $2n^2$ + 100n** to solving problem of size **n**.

6-6. Asymptotic Analysis

Asymptotic analysis is an analysis of algorithms that focuses on analyzing problems of **large input size n**, considers **only the leading term** of the formula, and **ignores the coefficient** of the leading term.

We choose the leading term because the lower order terms contribute lesser to the overall cost as the input grows larger, e.g., for f(n) = $2n^2$ + 100n, we have:
f(1000) = 2*1000^2 + 100*1000 = 2.1M, vs
f(100000) = 2*100000^2 + 100*100000 = 20010M.
(notice that the lower order term 100n has lesser contribution).

6-7. Ignoring Coefficient of the Leading Term

Suppose two algorithms have $2n^2$ and $30n^2$ as the leading terms, respectively.

Although actual time will be different due to the different constants, the growth rates of the running time are the same.

Compared with another algorithm with leading term of n^3, the difference in growth rate is a much more dominating factor.

Hence, we can drop the coefficient of leading term when studying algorithm complexity.

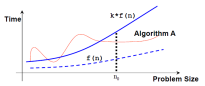
6-8. Upper Bound: The Big-O Notation

If algorithm A requires time proportional to **f(n)**, we say that algorithm A is of the order of f(n).

We write that algorithm A has time complexity of **O(f(n))**, where **f(n)** is the growth rate function for algorithm A.

6-9. Big-O Notation (Mathematics)

Mathematically, an algorithm A is of O(**f(n)**) if there exist a constant **k** and a positive integer **n0** such that algorithm A requires no more than **k*f(n)** time units to solve a problem of size **n ≥ n0**, i.e., when the problem size is larger than **n0**, then algorithm A is (always) bounded from above by this simple formula **k*f(n)**.



Note that: **n0** and **k** are not unique and there can be many possible valid **f(n)**.

6-10. Growth Terms

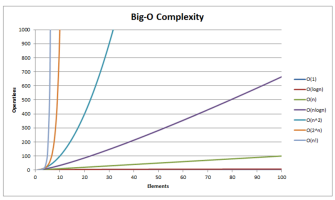
In asymptotic analysis, a formula can be simplified to a single term with coefficient 1.

Such a term is called a growth term (rate of growth, order of growth, order of magnitude).

The most common growth terms can be ordered from fastest to slowest as follows:
O(**1**)/constant time < O(log **n**)/logarithmic time < O(**n**)/linear time <
O(**n** log **n**)/quasilinear time < O($\mathbf{n}^2$)/quadratic time < O($\mathbf{n}^3$)/cubic time <
O($2^{\mathbf{n}}$)/exponential time < O(**n**!)/also-exponential time < ∞ (e.g., an infinite loop).

Note that a few other common time complexities are not shown (also see the visualization in the next slide).

6-11. Growth Terms (Visualized/Compared)



We will see three different growth rates O($\mathbf{n}^2$), O(**n log n**), and O(**n**) throughout the remainder of this sorting module.

7. Bubble Sort

Given an array of **N** elements, Bubble Sort will:

1. **Compare** a pair of adjacent items (a, b),
2. Swap that pair if the items are out of order (in this case, when a > b),
3. Repeat Step 1 and 2 until we reach the end of array
   (the last pair is the (**N**-2)-th and (**N**-1)-th items as we use 0-based indexing),
4. By now, the largest item will be at the last position.
   We then reduce **N** by 1 and repeat Step 1 until we have **N = 1**.

[This sentence is irrelevant for notes]

7-1. Bubble Sort, Pseudocode & Analysis

```
method bubbleSort(array A, integer N) // the standard version
  for each R from N-1 down to 1 // repeat for N-1 iterations
    for each index i in [0..R-1] // the 'unsorted region', O(N)
      if A[i] > A[i+1] // these two are not in non-decreasing order
        swap(a[i], a[i+1]) // swap them in O(1)
```

Comparison and swap require time that is bounded by a constant, let's call it **c**. Then, there are two nested loops in (the standard) Bubble Sort. The outer loop runs for exactly **N-1** iterations. But the inner loop runs get shorter and shorter:

1. When R=**N**-1, (**N**−1) iterations (of comparisons and possibly swaps),
2. When R=**N**-2, (**N**−2) iterations,
   …,
3. When R=1, 1 iteration (then done).

Thus, the total number of iterations = (**N**−1)+(**N**−2)+...+1 = **N***(**N**−1)/2 (derivation).

Total time = c*N*(**N**−1)/2 = O(N^2).

See the code shown in SortingDemo.cpp | py | java.

7-2. Bubble Sort: Early Termination

Bubble Sort is actually inefficient with its **O(N^2)** time complexity. Imagine that we have **N** = 10^5 numbers. Even if our computer is super fast and can compute 10^8 operations in 1 second, Bubble Sort will need about 100 seconds to complete.

However, it can be terminated early, e.g., on the small sorted ascending example shown above [3, 6, 11, 25, 39], Bubble Sort can terminates in O(**N**) time.

The improvement idea is simple: If we go through the inner loop with **no swapping** at all, it means that the array is **already sorted** and we can stop Bubble Sort at that point.

Discussion: Although it makes Bubble Sort runs faster in general cases, this improvement idea does not change **O(N^2)** time complexity of Bubble Sort... Why?

7-3. The Answer

[This is a hidden slide]

8. Selection Sort

Given an array of **N** items and **L** = 0, Selection Sort will:

1. Find the position **X** of the smallest item in the range of [**L**...**N**−1],
2. Swap **X**-th item with the **L**-th item,
3. Increase the lower-bound **L** by 1 and repeat Step 1 until **L** = **N**-2.

[This sentence is irrelevant for notes]

Without loss of generality, we can also implement Selection Sort in reverse:
Find the position of the largest item **Y** and swap it with the last item.

8-1. Selection Sort, Pseudocode & Analysis

```
method selectionSort(array A[], integer N)
  for each L in [0..N-2] // O(N)
    let X be the index of the minimum element in A[L..N-1] // O(N)
    swap(A[X], A[L]) // O(1), X may be equal to L (no actual swap)
```

Total: O(N^2) — To be precise, it is similar to Bubble Sort analysis.

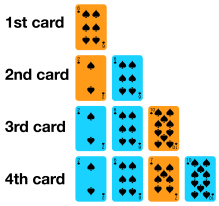
See the code shown in SortingDemo.cpp | py | java.

8-2. Mini Quiz

[This sentence is irrelevant for notes]

9. Insertion Sort

Insertion sort is similar to how most people arrange a hand of poker cards.



1. Start with one card in your hand,
2. Pick the next card and insert it into its proper sorted order,
3. Repeat previous step for all cards.

[This sentence is irrelevant for notes]

9-1. Insertion Sort, Pseudocode and Analysis 1

```
method insertionSort(array A[], integer N)
  for i in [1..N-1] // O(N)
    let X be A[i] // X is the next item to be inserted into A[0..i-1]
    for j from i-1 down to 0 // this loop can be fast or slow
      if A[j] > X
        A[j+1] = A[j] // make a place for X
      else
        break
    A[j+1] = X // insert X at index j+1
```

See the code shown in SortingDemo.cpp | py | java.

9-2. Insertion Sort: Analysis 2

The outer loop executes **N**−1 times, that's quite clear.

But the number of times the inner-loop is executed depends on the input:

1. In best-case scenario, the array is already sorted and (a[j] > X) is always false
   So no shifting of data is necessary and the inner loop runs in O(**1**),
2. In worst-case scenario, the array is reverse sorted and (a[j] > X) is always true
   Insertion always occur at the front of the array and the inner loop runs in O(**N**).

Thus, the best-case time is O(**N** × **1**) = O(**N**) and the worst-case time is O(**N** × **N**) = O($\mathbf{N}^2$).

9-3. Mini Quiz

[This sentence is irrelevant for notes]

10. 2.5 O(N log N) Comparison-based Sorting

We will discuss two (and a half) comparison-based sorting algorithms soon:

1. Merge Sort,
2. Quick Sort and its Randomized version (which only has one change).

These sorting algorithms are usually implemented recursively, use Divide and Conquer problem solving paradigm, and run in O(**N** log **N**) time for Merge Sort and O(**N** log **N**) time *in expectation* for Randomized Quick Sort.

PS: The non-randomized version of Quick Sort runs in O($\mathbf{N}^2$) though.

11. Merge Sort

Given an array of **N** items, Merge Sort will:

1. Merge each pair of individual element (which is by default, sorted) into sorted arrays of 2 elements,
2. Merge each pair of sorted arrays of 2 elements into sorted arrays of 4 elements,
   Repeat the process…,
3. Final step: Merge 2 sorted arrays of **N**/2 elements (for simplicity of this discussion, we assume that **N** is even) to obtain a fully sorted array of **N** elements.

This is just the general idea and we need a few more details before we can discuss the true form of Merge Sort.

11-1. Important Subroutine, O(N) Merge

We will dissect this Merge Sort algorithm by first discussing its most important sub-routine: The O(**N**) `merge`.

Given two sorted array, A and B, of size $\mathbf{N_1}$ and $\mathbf{N_2}$, we can efficiently merge them into one larger combined sorted array of size **N** = $\mathbf{N_1}$+$\mathbf{N_2}$, in O(**N**) time.

This is achieved by simply comparing the front of the two arrays and take the smaller of the two at all times. However, this simple but fast O(**N**) `merge` sub-routine will need additional array to do this merging correctly.

11-2. Merge Subroutine Pseudocode Implementation

```
method merge(array A, integer low, integer mid, integer high)
  // subarray1 = a[low..mid], subarray2 = a[mid+1..high], both sorted
  int N = high-low+1
  create array B of size N // discuss: why do we need a temp array b?
  int left = low, right = mid+1, bIdx = 0
  while (left <= mid && right <= high) // the merging
    if (A[left] <= A[right])
      B[bIdx++] = A[left++]
    else
      B[bIdx++] = A[right++]
  while (left <= mid)
    B[bIdx++] = A[left++] // leftover, if any
  while (right <= high)
    B[bIdx++] = A[right++] // leftover, if any
  for (int k = 0; k < N; ++k)
    A[low+k] = B[k]; // copy back
```

[This sentence is irrelevant for notes]

11-3. Divide and Conquer Paradigm

Before we continue, let's talk about Divide and Conquer (abbreviated as D&C), a powerful problem solving paradigm.

Divide and Conquer algorithm solves (certain kind of) problem — like our sorting problem — in the following steps:

1. Divide step: Divide the large, original problem into smaller sub-problems and recursively solve the smaller sub-problems,
2. Conquer step: Combine the results of the smaller sub-problems to produce the result of the larger, original problem.

11-4. Merge Sort as a D&C Algorithm

Merge Sort is a Divide and Conquer sorting algorithm.

The divide step is simple: Divide the current array into two halves (perfectly equal if **N** is even or one side is slightly greater by one element if **N** is odd) and then recursively sort the two halves.

The conquer step is the one that does the most work: Merge the two (sorted) halves to form a sorted array, using the merge sub-routine discussed earlier.

11-5. Merge Sort Pseudocode Implementation

```
method mergeSort(array A, integer low, integer high)
  // the array to be sorted is A[low..high]
  if (low < high) // base case: low >= high (0 or 1 item)
    int mid = (low+high) / 2
    mergeSort(a, low  , mid ) // divide into two halves
    mergeSort(a, mid+1, high) // then recursively sort them
    merge(a, low, mid, high) // conquer: the merge subroutine
```

See the code shown in SortingDemo.cpp | py | java.

11-6. Demonstration

Contrary to what many other CS printed textbooks usually show (as textbooks are static), the actual execution of Merge Sort does **not** split to two subarrays **level by level**, but it will recursively sort the **left** subarray first before dealing with the **right** subarray.

That's it, running Merge Sort on the example array [7, 2, 6, 3, 8, 4, 5], it will recurse to [7, 2, 6, 3], then [7, 2], then [7] (a single element, sorted by default), backtrack, recurse to [2] (sorted), backtrack, then finally merge [7, 2] into [2, 7], before it continue processing [6, 3] and so on.

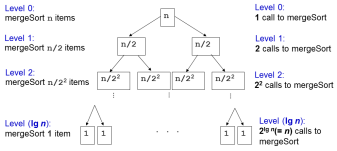
11-7. Merge Sort: Analysis Part 1

In Merge Sort, the bulk of work is done in the conquer/merge step as the divide step does not really do anything (treated as O(**1**)).

When we call `merge(a, low, mid, high)`, we process **k = (high-low+1)** items.
There will be at most **k-1** comparisons.
There are **k** moves from original array **a** to temporary array **b** and another **k** moves back.
In total, number of operations inside `merge` sub-routine is < 3**k**-1 = O(**k**).

The important question is how many times this `merge` sub-routine is called?

11-8. Merge Sort: Analysis Part 2



$n/(2^k) = 1 \rightarrow n = 2^k \rightarrow k = \lg n$

11-9. Merge Sort: Analysis Part 3

Level 1: 2^0=1 calls to merge() with **N**/2^1 items each, O(2^0 x 2 x **N**/2^1) = O(**N**)
Level 2: 2^1=2 calls to merge() with **N**/2^2 items each, O(2^1 x 2 x **N**/2^2) = O(**N**)
Level 3: 2^2=4 calls to merge() with **N**/2^3 items each, O(2^2 x 2 x **N**/2^3) = O(**N**)
...
Level (log **N**): 2^(log **N**-1) (or **N**/2) calls to merge() with **N**/2^log **N** (or 1) item each, O(**N**)

There are log **N** levels and in each level, we perform O(**N**) work, thus the overall time complexity is O(**N** log **N**). We will later see that this is an optimal (comparison-based) sorting algorithm, i.e., we cannot do better than this.

11-10. Pros and Cons

The most important good part of Merge Sort is its O(**N** log **N**) performance guarantee, regardless of the original ordering of the input. That's it, there is **no** adversary test case that can make Merge Sort runs longer than O(**N** log **N**) for **any** array of **N** elements.

Merge Sort is therefore very suitable to sort extremely large number of inputs as O(**N** log **N**) grows much slower than the $O(N^2)$ sorting algorithms that we have discussed earlier.

There are however, several not-so-good parts of Merge Sort. First, it is actually not easy to implement from scratch (but we don't have to). Second, it requires additional O(**N**) storage during merging operation, thus not really memory efficient and not in-place. Btw, if you are interested to see what have been done to address these (classic) Merge Sort not-so-good parts, you can read this.

Merge Sort is also a stable sort algorithm. Discussion: Why?

11-11. The Answer

[This is a hidden slide]

12. Quick Sort

Quick Sort is another Divide and Conquer sorting algorithm (the other one discussed in this visualization page is Merge Sort).

We will see that this deterministic, non randomized version of Quick Sort can have bad time complexity of $O(N^2)$ on adversary input before continuing with the randomized and usable version later.

12-1. Quick Sort as a D&C Algorithm

Divide step: Choose an item **p** (known as the pivot)
Then partition the items of **A[i..j]** into three parts: **A[i..m-1]**, **A[m]**, and **A[m+1..j]**.
A[i..m-1] (possibly empty) contains items that are smaller than (or equal to) **p**.
A[m] = p, i.e., index **m** is the correct position for **p** in the sorted order of array **a**.
A[m+1..j] (possibly empty) contains items that are greater than (or equal to) **p**.
Then, recursively sort the two parts.

Conquer step: Don't be surprised... We do nothing :O!

If you compare this with Merge Sort, you will see that Quick Sort D&C steps are totally opposite with Merge Sort.

12-2. Important Sub-routine, O(N) Partition

We will dissect this Quick Sort algorithm by first discussing its most important sub-routine: The O(**N**) `partition` (classic version).

To partition **A[i..j]**, we first choose **A[i]** as the pivot **p**.

The remaining items (i.e., **A[i+1..j]**) are divided into 3 regions:

1. **S1** = **A[i+1..m]** where items are ≤ **p**,
2. **S2** = **A[m+1..k-1]** where items are ≥ **p**, and
3. Unknown = **A[k..j]**, where items are yet to be assigned to either **S1** or **S2**.

Discussion: Why do we choose **p** = **A[i]**? Are there other choices?

Harder Discussion: If **A[k] == p**, should we put it in region S1 or S2?

12-3. The Answer

[This is a hidden slide]

12-4. Partition - Continued

Initially, both **S1** and **S2** regions are empty, i.e., all items excluding the designated pivot **p** are in the unknown region.

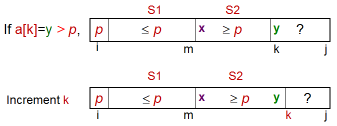
Then, for each item **A[k]** in the unknown region, we compare **A[k]** with **p** and decide one of the three cases:

1. If **A[k]** > **p**, put **A[k]** into **S2**,
2. If **A[k]** < **p**, put **A[k]** into **S1**,
3. If **A[k] == p**, throw a coin and put **A[k]** into **S1**/**S2** if it lands head/tail, respectively.

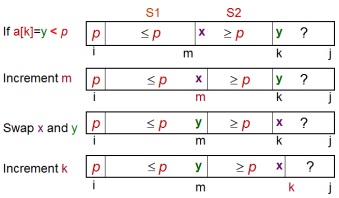
These three cases are elaborated in the next two slides.

Lastly, we swap **A[i]** and **A[m]** to put pivot **p** right in the middle of **S1** and **S2**.

12-5. Partition - Case when A[k] > p



12-6. Partition - Case when A[k] < p



12-7. Partition Pseudocode Implementation

```
int partition(array A, integer i, integer j)
  int p = a[i] // p is the pivot
  int m = i // S1 and S2 are initially empty
  for (int k = i+1; k <= j; ++k) // explore the unknown region
    if ((A[k] < p) || ((A[k] == p) && (rand()%2 == 0)))  { // case 2+3
      ++m
      swap(A[k], A[m]) // exchange these two indices
      // notice that we do nothing in case 1: A[k] > p
  swap(A[i], A[m]) // final step, swap pivot with a[m]
  return m // return the index of pivot
```

12-8. Quick Sort Pseudocode Implementation

```
method quickSort(array A, integer low, integer high)
  if (low < high)
    int m = partition(a, low, high) // O(N)
    // A[low..high] ~> A[low..m-1], pivot, A[m+1..high]
    quickSort(A, low, m-1); // recursively sort left subarray
    // A[m] = pivot is already sorted after partition
    quickSort(A, m+1, high); // then sort right subarray
```

See the code shown in SortingDemo.cpp | py | java.

12-9. Demonstration

Try Quick Sort on example array [27, 38, 12, 39, 29, 16]. We shall elaborate the first partition step as follows:
We set **p = A[0] = 27**.
We set **A[1] = 38** as part of **S2** so **S1 = {}** and **S2 = {38}**.

We swap **A[1] = 38** with **A[2] = 12** so **S1 = {12}** and **S2 = {38}**.
We set **A[3] = 39** and later **A[4] = 29** as part of **S2** so **S1 = {12}** and **S2 = {38,39,29}**.
We swap **A[2] = 38** with **A[5] = 16** so **S1 = {12,16}** and **S2 = {39,29,38}**.
We swap **p = A[0] = 27** with **A[2] = 16** so **S1 = {16,12}**, **p = {27}**, and **S2 = {39,29,38}**.

After this, **A[2] = 27** is guaranteed to be sorted and now Quick Sort recursively sorts the left side **A[0..1]** first and later recursively sorts the right side **A[3..5]**.

12-10. Quick Sort: Analysis Part 1

First, we analyze the cost of one call of `partition`.

Inside `partition(A, i, j)`, there is only a single for-loop that iterates through (j-i) times. As j can be as big as **N**-1 and i can be as low as 0, then the time complexity of partition is O(**N**).

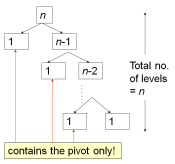
Similar to Merge Sort analysis, the time complexity of Quick Sort is then dependent on the number of times `partition(A, i, j)` is called.

12-11. Quick Sort: Analysis Part 2

When the array **A** is already in ascending order, e.g., **A** = [5, 18, 23, 39, 44, 50], Quick Sort will set **p = A[0] = 5**, and will return **m = 0**, thereby making **S1** region **empty** and **S2** region: Everything else other than the pivot (**N**-1 items).

12-12. Quick Sort: Analysis Part 3

On such worst case input scenario, this is what happens:



The first partition takes O(**N**) time, splits **A** into 0, 1, **N**-1 items, then recurse right.
The second one takes O(**N**-1) time, splits **A** into 0, 1, **N**-2 items, then recurse right again.
...
Until the last, **N**-th partition splits **A** into 0, 1, 1 item, and Quick Sort recursion stops.

This is the classic **N+(N-1)+(N-2)+...+1** pattern, which is $O(N^2)$, similar analysis as the one in this Bubble Sort analysis slide...

12-13. Quick Sort: Best Case (Rare)

The best case scenario of Quick Sort occurs when partition always splits the array into **two equal halves**, like Merge Sort.

When that happens, the depth of recursion is only O(log **N**).

As each level takes O(**N**) comparisons, the time complexity is O(**N** log **N**).

Try Quick Sort on this hand-crafted example input array [4, 1, 3, 2, 6, 5, 7].
In practice, this is rare, thus we need to devise a better way: Randomized Quick Sort.

13. Random Quick Sort

Same as **Quick Sort** except just before executing the partition algorithm, it **randomly** select the pivot between **A[i..j]** instead of always choosing **A[i]** (or any other fixed index between **[i..j]**) deterministically.

Mini exercise: Implement the idea above to the implementation shown in this slide!

Running Random Quick Sort on this large and somewhat random example array **a** = [3,44,38,5,47,15,36,26,27,2,46,4,19,50,48] feels fast.

13-1. Magical Analysis

It will take about 1 hour lecture to properly explain why this randomized version of Quick Sort has expected time complexity of O(**N** log **N**) on **any** input array of **N** elements.

In this e-Lecture, we will assume that it is true.

If you need non formal explanation: Just imagine that on randomized version of Quick Sort that randomizes the pivot selection, we will **not** always get extremely bad split of 0 (empty), 1 (pivot), and **N**-1 other items. This combination of lucky (half-pivot-half), somewhat lucky, somewhat unlucky, and extremely unlucky (empty, pivot, the rest) yields an average time complexity of O(**N** log **N**).

Discussion: For the implementation of Partition, what happen if **A[k] == p**, we *always* put **A[k]** on either side (**S1** or **S2**) deterministically?

13-2. The Answer

[This is a hidden slide]

14. 2 O(N) Non Comparison-based Sorting Algorithms

We will discuss two **non comparison-based** sorting algorithms in the next few slides:

1. Counting Sort,
2. Radix Sort.

These sorting algorithms can be faster than the lower bound of comparison-based sorting algorithm of Ω(**N** log **N**) by **not** comparing the items of the array.

14-1. Lower Bound of Sorting Algorithm

It is known (also not proven in this visualization as it will take about half-an-hour lecture about decision tree model to do so) that all **comparison-based** sorting algorithms have a lower bound time complexity of Ω(**N** log **N**).

Thus, any comparison-based sorting algorithm with worst-case complexity O(**N** log **N**), like Merge Sort is considered an optimal algorithm, i.e., we cannot do better than that.

However, we can achieve faster sorting algorithm — i.e., in O(**N**) — if certain assumptions of the input array exist and thus we can avoid comparing the items to determine the sorted order.

15. Counting Sort

Assumption: If the items to be sorted are **Integers with small range**, we can count the frequency of occurrence of each Integer (in that small range) and then loop through that small range to output the items in sorted order.

Try Counting Sort on the example array above where all Integers are within [1..9], thus we just need to count how many times Integer 1 appears, Integer 2 appears, ..., Integer 9 appears, and then loop through 1 to 9 to print out **x** copies of Integer **y** if frequency[**y**] = **x**.

The time complexity is O(**N**) to count the frequencies and O(**N**+**k**) to print out the output in sorted order where **k** is the range of the input Integers, which is 9-1+1 = 9 in this example. The time complexity of Counting Sort is thus O(**N**+**k**), which is O(**N**) if **k** is small.

We will not be able to do the counting part of Counting Sort when **k** is relatively big due to memory limitation, as we need to store frequencies of those **k** integers.

PS: This version of Counting Sort is not stable, as it does not actually remember the (input) ordering of duplicate integers. The version presented in CLRS is stable, but is a bit more complex than this form.

16. Radix Sort

Assumption: If the items to be sorted are **Integers with large range but of few digits**, we can combine Counting Sort idea with Radix Sort to achieve the linear time complexity.

In Radix Sort, we treat each item to be sorted as a string of **w** digits (we pad Integers that have less than **w** digits with leading zeroes if necessary).

For the least significant (rightmost) digit to the most significant digit (leftmost), we pass through the **N** items and put them according to the active digit into 10 Queues (one for each digit [0..9]), which is like a *modified* Counting Sort as this one preserves stability (remember, the Counting Sort version shown in this slide earlier is not a stable sort). Then we re-concatenate the groups again for subsequent iteration.

Try Radix Sort on the random 4-digits array above for clearer explanation.

Notice that we only perform O(**w** × (**N**+**k**)) iterations. In this example, **w = 4** and **k = 10**.

16-1. The Best Sorting Algorithm for Integers?

Now, having discussed about Radix Sort, should we use it for **every** sorting situation?

For example, it should be theoretically faster to sort many (**N** is very large) 32-bit signed integers as **w ≤ 10** digits and **k = 10** if we interpret those 32-bit signed integers in Decimal. O(10 × (**N**+10)) = O(**N**).

Discussion: Using base-10 as shown in this visualization is actually not the best way to sort **N** 32-bit signed integers. What should be the better setup?

16-2. The Answer

[This is a hidden slide]

17. Additional Properties of Sorting Algorithms

There are a few other properties that can be used to differentiate sorting algorithms on top of whether they are comparison or non-comparison, recursive or iterative.

In this section, we will talk about in-place versus not in-place, stable versus not stable, and caching performance of sorting algorithms.

17-1. In-Place Sorting

A sorting algorithm is said to be an **in-place sorting** algorithm if it requires only a constant amount (i.e., O(**1**)) of extra space during the sorting process. That's it, a few, constant number of extra variables is OK but we are not allowed to have variables that has variable length depending on the input size **N**.

Merge Sort (the classic version), due to its `merge` sub-routine that requires additional temporary array of size **N**, is not in-place.

Discussion: How about Bubble Sort, Selection Sort, Insertion Sort, Quick Sort (randomized or not), Counting Sort, and Radix Sort. Which ones are in-place?

17-2. Stable Sort

A sorting algorithm is called **stable** if the relative order of elements **with the same key value** is preserved by the algorithm after sorting is performed.

Example application of stable sort: Assume that we have student names that have been sorted in alphabetical order. Now, if this list is sorted again by tutorial group number (recall that one tutorial group usually has many students), a stable sort algorithm would ensure that all students in the same tutorial group still appear in alphabetical order of their names. Radix sort that goes through multiple round of sorts digit-by-digit requires a stable sort sub-routine for it to work correctly.

Discussion: Which of the sorting algorithms discussed in this e-Lecture are stable?
Try sorting array A = {3, 4a, 2, 4b, 1}, i.e. there are two copies of 4 (4a first, then 4b).

17-3. Caching Performance

[This is a hidden slide]

18. Quizzes

We are nearing the end of this e-Lecture.

Time for a few simple questions.

18-1. Quiz #1

Quiz: **Which of these algorithms run in O(N log N) on any input array of size N?**

○ Quick Sort (Deterministic)
○ Bubble Sort
○ Merge Sort
○ Insertion Sort

18-2. Quiz #2

Quiz: **Which of these algorithms has worst case time complexity of Θ(N^2) for sorting N integers?**

☐ Insertion Sort
☐ Bubble Sort
☐ Radix Sort
☐ Merge Sort
☐ Selection Sort

Θ is a tight time complexity analysis where the best case Ω and the worst case big-O analysis match.

19. Extras

We have reached the end of sorting e-Lecture.

However, there are two other sorting algorithms in VisuAlgo that are embedded in other data structures: Heap Sort and Balanced BST Sort. We will discuss them when you go through the e-Lecture of those two data structures.

19-1. Challenge

[This is a hidden slide]

19-2. Inversion Index/Count

[This is a hidden slide]

19-3. Implementation

Actually, the C++ source code for many of these basic sorting algorithms are already scattered throughout these e-Lecture slides. For other programming languages, you can translate the given C++ source code to the other programming language.

Usually, sorting is just a small part in problem solving process and nowadays, most of programming languages have their own sorting functions so we don't really have to re-code them *unless absolutely necessary*.

In C++, you can use std::sort (most likely a hybrid sorting algorithm: Introsort), std::stable_sort (most likely Merge Sort), or std::partial_sort (most likely Binary Heap) in STL algorithm.
In Python, you can use sort (most likely a hybrid sorting algorithm: Timsort).
In Java, you can use Collections.sort.
In OCaml, you can use List.sort compare list_name.

If the comparison function is problem-specific, we may need to supply additional comparison function to those built-in sorting routines.

19-4. Online Quiz

Now it is time for you to see if you have understand the basics of various sorting algorithms discussed so far.

Test your understanding here.

19-5. Online Judge Exercises

Now that you have reached the end of this e-Lecture, do you think sorting problem is just as simple as calling built-in sort routine?

Try these online judge problems to find out more:
Kattis - mjehuric
Kattis - sortofsorting, or
Kattis - sidewaysorting

This is not the end of the topic of sorting. When you explore other topics in VisuAlgo, you will realise that sorting is a pre-processing step for many other advanced algorithms for harder problems, e.g. as the pre-processing step for Kruskal's algorithm, creatively used in Suffix Array data structure, etc.

20. Detailed Analysis of Randomized Quicksort

start of 3230 material

LL, Stack, Queue, DLL, Deque

1. Introduction

Linked List is a data structure consisting of a group of vertices (nodes) which together represent a sequence. Under the simplest form, each vertex is composed of a data and a reference (link) to the next vertex in the sequence. Try clicking Search(77) for a sample animation on searching a value in a (Singly) Linked List.

Linked List (and its variations) can be used as the underlying data structure to implement List, Stack, Queue, and Deque ADTs (read this Wikipedia article about ADT if you are not familiar with that term). Another potential data structure that can be used to implement List ADT is (resize-able) array.

In this visualization, we discuss (Singly) Linked List (LL) — with a single next pointer — and its two variants: Stack and Queue, and also Doubly Linked List (DLL) — with both next and previous pointers — and its variant: Deque.

1-1. Five Modes

We decide to group five related modes involving Linked List (LL, Stack, Queue, DLL, Deque) in one single visualization page. To facilitate more diversity, we randomize the selected mode upon loading this direct URL: https://visualgo.net/en/list.

However, you can use the following URL shortcuts to access individual mode directly (only works for logged-in users who have cleared reading all 3 sectors of these lecture notes):

1. https://visualgo.net/en/ll,
2. https://visualgo.net/en/stack,
3. https://visualgo.net/en/queue,
4. https://visualgo.net/en/dll,
5. https://visualgo.net/en/deque.

2. Motivation

Linked List data structure is commonly taught in Computer Science (CS) undergraduate courses for a few reasons:

1. It is a simple linear data structure,
2. It has a range of potential applications as a list ADT e.g., student list, event list, appointment list, etc (albeit there are other more advanced data structures that can do the same (and more) applications better) or as stack/queue/deque ADTs,
3. It has interesting corner/special cases to illustrate the need for a good implementation of a data structure,
4. It has various customization options and thus usually this Linked List data structure is taught using Object-Oriented Programming (OOP) way.

2-1. List ADT

List is a sequence of items/data where positional order matter $\{a_0, a_1, ..., a_{N-2}, a_{N-1}\}$.
Common List ADT operations are:

0. `get(i)` — maybe a trivial operation, return a_i (0-based indexing),
1. `search(v)` — decide if item/data **v** exists (and report its position/index) or not exist (and usually report a non existing index -1) in the list,
2. `insert(i, v)` — insert item/data **v** specifically at position/index **i** in the list, potentially shifting the items from previous positions: [**i**..**N**-1] by one position to their right to make a space,
3. `remove(i)` — remove item that is specifically at position/index **i** in the list, potentially shifting the items from previous positions: [**i+1**..**N**-1] by one position to their left to close the gap.

Discussion 1: What if we want to remove item with specific value **v** in the list?

Discussion 2: Can a List ADT contains duplicate items, i.e., $a_i = a_j$ where $i \neq j$?

2-2. The Answer

[This is a hidden slide]

2-3. Array Implementation (Pros and Cons)

Please review (Compact) Array data structure that can also be used to implement List ADT.

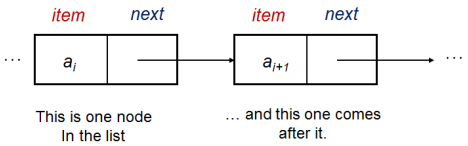
For fixed-size collections with known max limit of number of items that will ever be needed, i.e., the max size of **M**, then array is already a reasonably good data structure for List ADT implementation.

For variable-size collections with unknown size **M** and where dynamic operations such as insert/remove are common, a simple (fixed-size) array is actually a poor choice of data structure.

For such applications, there are better data structures, including Linked List. Read on...

3. Linked List (LL)

We now introduce the Linked List data structure. It uses pointers/references to allow items/data to be **non-contiguous** in memory (that is the main difference with a simple array). The items are ordered from index 0 to index **N**-1 by associating item **i** with its neighbour item **i+1** through a pointer.



3-1. Linked List Vertex C++ Implementation

In its basic form, a single vertex (node) in the Linked List has this rough structure:

```
struct Vertex { // we can use either C struct or C++/Python/Java class
  int item; // the data is stored here, an integer in this example
  Vertex* next; // this pointer tells us where is the next vertex
};
```

Using the default example Linked List [22 (head)->2->77->6->43->76->89 (tail)] for illustration, we have:
$\mathbf{a_0}$ with its **item** = 22 and its **next** = $\mathbf{a_1}$,
$\mathbf{a_1}$ with its **item** = 2 and its **next** = $\mathbf{a_2}$,
...
$\mathbf{a_6}$ with its **item** = 89 and its **next** = **null**.

Discussion: Which one is better for a C++ implementation of Linked List? struct or class? How about Python or Java implementation?

3-2. The Answer

[This is a hidden slide]

3-3. Linked List, Additional Data

We also have a few additional data that we remember in this Linked List data structure. We use the default example Linked List [22 (head)->2->77->6->43->76->89 (tail)] for illustration.

1. The **head** pointer points to $\mathbf{a_0}$ — it is 22, nothing points to the head item,
2. The current number of elements **N** in the Linked List — **N = 7** elements.
3. The **tail** pointer points to $\mathbf{a_{N-1}}$ — it is $\mathbf{a_6}$ = 89, nothing is after the tail item.

That's it, we only add three more extra variables in data structure.

3-4. Variations

Note that there are various subtle differences found in many Computer Science textbooks on how to implement a (Singly) Linked List e.g., use tail pointer or not, circular or not, use dummy head item or not, allow duplicate items or not — see this slide.

Our version in this visualization (with tail pointer, not circular, without dummy head, disallow duplicate) may not be 100% the same compared to what you learn in your class but the basic ideas should remain the same.

In this visualization, each vertex has Integer item, but this can easily be changed to any other data type as needed.

3-5. Get(i) - Much Slower than Array

Since we only keep the head and tail pointers, list traversal subroutine is needed to reach positions other than the head (index 0) and the tail (index **N**-1).

As this sub-routine is so frequently used, we will abstract it out as a function. The code below is written in C++.

```
Vertex* Get(int i) { // returns the vertex
  Vertex* ptr = head; // we have to start from head
  for (int k = 0; k < i; ++k) // advance forward i time(s)
    ptr = ptr->next; // the pointers are pointing to the higher index
  return ptr;
}
```

It runs in O(**N**) as **i** can be as big as index **N**-2.
Compare this with array where we can access index **i** in O(**1**) time.

3-6. Search(v) - Not Better than Array

As we only have direct reference to the first head item and the last tail item, plus the pointers are pointing to the **right** (higher position/index), we can only access the rest by starting from the head item and hopping through the next pointers. On the default [22 (head)->2->77->6->43->76->89 (tail)], we have:

Search(77) — found in the example above at position/index 2 (0-based indexing).

Search(7) — not found in the example above, and this is only known after all **N** items are examined, so `Search(v)` has O(**N**) worst case time complexity.

3-7. Insertion - Four Cases

There are more cases than array version due to the nature of Linked List.

Most CS students who learn Linked List for the first time usually are not aware of all cases until they figure it out themselves when their Linked List code fail.

In this c-Lecture, we directly elaborate all cases.

For `insert(i, v)`, there are four (legal) possibilities, i.e., item **v** is added to:

1. The head (before the current first item) of the linked list, **i = 0**,
2. An empty linked list (which fortunately similar to the previous case),
3. The position beyond the last (the current tail) item of the linked list, **i = N**,
4. The other positions of the linked list, **i = [1..N-1]**.

3-8. Insert(i, v) - Insert at Head (i = 0)

The (C++) code for insertion at head is simple and efficient, in O(**1**):

```
Vertex* vtx = new Vertex(); // create new vertex vtx from item v
vtx->item = v;
vtx->next = head; // link this new vertex to the (old) head vertex
head = vtx; // the new vertex becomes the new head
```

Try InsertHead(50), which is `insert(0, 50)`, on the example Linked List [22 (head)->2->77->6->43->76->89 (tail)].

Discussion: What happen if we use array implementation for insertion at head of the list?

3-9. The Answer

[This is a hidden slide]

3-10. Insert(i, v) - Insert into an Empty List

Empty data structure is a common corner/special case that can often cause unexpected crash if not properly tested. It is legal to insert a new item into a currently empty list, i.e., at index **i = 0**. Fortunately, the same pseudo-code for insertion at head also works for an empty list so we can just use the same code as in this slide (with one minor change, we also need to set tail = head).

Try InsertHead(50), which is `insert(0, 50)`, but on the empty Linked List [].

3-11. Insert(i, v) - In Between, i ∈ [1..N-1]

With the Linked List traversal `Get(i)` sub-routine, we can now implement insertion in the middle of the Linked List as follows (in C++):

```
Vertex* pre = Get(i-1); // traverse to (i-1)-th vertex, O(N)
aft = pre->next; // aft cannot be null, think about it
Vertex* vtx = new Vertex(); // create new vertex
vtx->item = v;
vtx->next = aft; // link this
pre->next = vtx; // and this
```

Try Insert(3, 44) on the example Linked List [22 (head)->2->77->6->43->76->89 (tail)].

Also try Insert(6, 55) on the same example Linked List. This is a corner case: Insert at the position of tail item, shifting the tail to one position to its right.

This operation is slow, O(**N**), due to the need for traversing the list (e.g. if **i** close to **N**-1).

3-12. Insert(i, v) - Beyond the Tail, i = N

If we also remember the tail pointer as with the implementation in this c-Lecture (which is advisable as it is just one additional pointer variable), we can perform insertion beyond the tail item (at **i = N**) efficiently, in O(**1**):

```
Vertex* vtx = new Vertex(); // this is also a C++ code
vtx->item = v; // create new vertex vtx from item v
tail->next = vtx; // just link this, as tail is the i = (N-1)-th item
tail = vtx; // now update the tail pointer
```

Try InsertTail(10), which is `insert(7, 10)`, on the example Linked List [22 (head)->2->77->6->43->76->89 (tail)] . A common misconception is to say that this is insertion at tail. Insertion at tail element is `insert(N-1, v)`. Insertion **beyond** the tail is `insert(N, v)`.

Discussion: What happen if we use array implementation for insertion beyond the tail of the list?

3-13. The Answer

[This is a hidden slide]

3-14. Removal - Three Cases

For `remove(i)`, there are three (legal) possibilities, i.e., index **i** is:

1. The head (the current first item) of the linked list, **i = 0**, it affects the head pointer
2. The tail of the linked list, **i = N-1**, it affects the tail pointer
3. The other positions of the linked list, **i = [1..N-2]**.

Discussion: Compare this slide with Insertion Cases slide to realize the subtle differences. Is removing anything from an already empty Linked List considered 'legal'?

3-15. Remove(i) - At Head (i = 0)

This case is straightforward (written in C++):

```
if (head == NULL) return; // avoid crashing when SLL is empty
Vertex* tmp = head; // so we can delete it later
head = head->next; // book keeping, update the head pointer
delete tmp; // which is the old head
```

Try RemoveHead() repeatedly on the (shorter) example Linked List [22 (head)->2->77->6 (tail)]. It will continuously working correctly up until the Linked List contains one item where the head = the tail item. We prevent execution if the LL is already empty as it is an illegal case.

Discussion: What happen if we use array implementation for removal of head of the list?

3-16. The Answer

[This is a hidden slide]

3-17. Remove(i) - In Between, i ∈ [1..N-2]

With the Linked List traversal `Get(i)` sub-routine (discussed earlier), we can now implement removal of the middle item of the Linked List as follows (in C++):

```
Vertex* pre = Get(i-1); // traverse to (i-1)-th vertex, O(N)
Vertex* del = pre->next, aft = del->next;
pre->next = aft; // bypass del
delete del;
```

Try Remove(5), the element at index **N-2** (as **N = 7** in the example [22 (head)->2->77->6->43->76->89 (tail)] .
This is the worst O(**N**) case on the example above.

Note that **Remove(N-1)** is removal at tail that requires us to update the tail pointer, see the next case.

3-18. Remove(i) - At Tail (i = N-1) - Part 1

We can implement the removal of the tail of the Linked List as follows, assuming that the Linked List has more than 1 item (in C++):

```
Vertex* pre = head;
tmp = head->next;
while (tmp->next != null) // while my neighbor is not the tail
  pre = pre->next, tmp = tmp->next;
pre->next = null;
delete tmp; // tmp = (old) tail
tail = pre; // update tail pointer
```

Try RemoveTail() repeatedly on the (shorter) example Linked List [22 (head)->2->77->6 (tail)]. It will continuously working correctly up until the Linked List contains one item where the head = the tail item and we switch to removal at head case. We prevent execution if the LL is already empty as it is an illegal case.

3-19. Remove(i) - At Tail (i = N-1) - Part 2

Actually, if we also maintain the size of the Linked List **N** (compare with this slide), we can use the Linked List traversal sub-routine `Get(i)` to implement the removal of the tail of the Linked List this way (in C++):

```
Vertex* pre = Get(N-2); // go to one index just before tail, O(N)
pre->next = null;
delete tail;
tail = pre; // we have access to old tail
```

Notice that this operation is slow, O(**N**), just because of the need to update the tail pointer from item **N**-1 backwards by one unit to item **N**-2 so that future insertion after tail remains correct... This deficiency will be later addressed in Doubly Linked List variant.

Discussion: What happen if we use array implementation for removal of tail of the list?

3-20. The Answer

[This is a hidden slide]

3-21. Time Complexity Summary

`get(i)` is slow: O(**N**).
In Linked List, we need to perform sequential access from head element.

`search(v)`
In the best case, **v** is found at the first position, O(**1**).

In the worst case, **v** is not found in the list and we require O(**N**) scan to determine that.

`insert(i, v)`
In the best case, insert at **i = 0** or at **i = N**, head and tail pointers help, O(**1**).
In the worst case, insert at **i = N-1**, we need to find the item **N**-2 just before the tail, O(**N**).

`remove(i)`
In the best case, remove at **i = 0**, head pointer helps, O(**1**).
In the worst case, remove at **i = N-1**, due to the need to update the tail pointer, O(**N**).

3-22. Linked List Applications

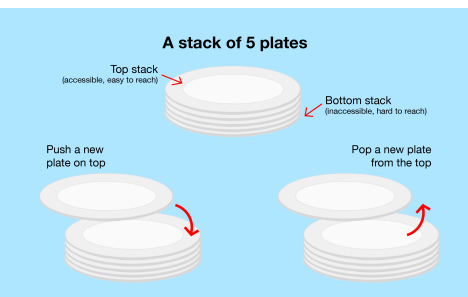
Pure (Singly) Linked List applications are surprisingly rare as the simpler resizeable compact array (vector) can do the job better, compare the Linked List version with the compact array version.

However, the basic concept of Linked List that allows the vertices to be non-contiguous in memory makes it an excellent resize-able data structure for the next two other Abstract Data Types: Stack and Queue.

4. Stack

Stack is a particular kind of Abstract Data Type in which the main operations on the collection are the addition of an item to the collection, known as push, only to the top of the stack and removal of an item, known as pop, only from the top of the stack.

It is known as Last-In-First-Out (LIFO) data structure, e.g., the stack of book below.



4-1. The Design Choice

In most implementations and also in this visualization, Stack is basically a **protected** (Singly) Linked List where we can only peek at the head item, push a new item only to the head (insert at head), e.g., try InsertHead(6), and pop existing item only from the head (remove from head), e.g., try RemoveHead(). All operations are O(1).

In this visualization, we orientate the (Single) Linked List top down, with the head/tail item at the top/bottom, respectively. In the example, we have [2 (top/head)->7->5->3->1->9 (bottom/tail)]. Due to vertical screen size limit (in landscape mode), we only allow a Stack of at most 7 items in this visualization.

Discussion: Can we use vector, a resizeable array, to implement Stack ADT efficiently?

4-2. The Answer

[This is a hidden slide]

4-3. Stack Applications

Stack has a few popular textbook applications. Some examples:

1. Bracket Matching,
2. Postfix Calculator,
3. A few other interesting applications that are not shown for pedagogical purposes.

4-4. Bracket Matching Problem

Mathematical expression can get quite convoluted, e.g., `{[x+2]^(2+5)-2}*(y+5)`.

Bracket Matching problem is a problem of checking whether all brackets in the given input are matched correctly, i.e., (with), [with] and { with }, and so on.

Bracket Matching is equally useful for checking the legality of a source code.

Discussion: It turns out that we can use Stack's LIFO behavior to solve this problem.
The question is how?

4-5. O(N) Solution with Stack

[This is a hidden slide]

4-6. Calculating Postfix Expression

Postfix expression is a mathematical expression in: `operand1 operand2 operator` format which is different from what human is most comfortable at, the Infix expression: `operand1 operator operand2`.

For example, expression `2 3 + 4 *` is the Postfix version of `(2+3)*4`.

In Postfix expression, we do not need brackets.

Discussion: It turns out that we can also use Stack to solve this problem efficiently.
The question is how?

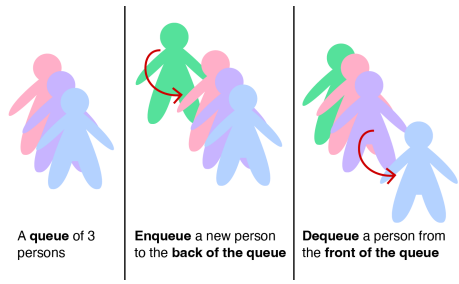
4-7. O(N) Solution with Stack

[This is a hidden slide]

5. Queue

Queue is another Abstract Data Type in which the items in the collection are kept **in order** and the main operations on the collection are the addition of items to the back position (enqueue) and removal of items from the front position (dequeue).

It is known as **First-In-First-Out (FIFO)** data structure as the first item to be enqueued will eventually be the first item to be dequeued, as in real life queues (see below).



5-1. Array Implementation Issues - Part 1

If we simply use the compact array implementation for this Queue ADT with $\mathbf{a_0}$ is the front of the queue and $\mathbf{a_{N-1}}$ is the back of the queue, we will encounter major performance issue with the **dequeue** operation.

This is because insertion at the back of a compact array is fast, O(**1**), but removal at the front of a compact array is slow due to the need to shift items, please review this slide.

5-2. Array Implementation Issues - Part 2

Another possible array implementation is to avoid that shifting of items during dequeue operation by having **two** indices: **front** (the index of the queue front-most item, increased after a dequeue operation) and **back** (the index of the queue back-most item, also increased after an enqueue operation).

Suppose we use an array of size **M = 8** items and the content of our queue is as follows: `[(2,4,1,7),-,-,-,-]` with **front = 0** (underlined) and ***back = 3*** *(italic)*. The current active queue elements are highlighted with (green color).

If we call dequeue(), we have `[-,(4,1,7),-,-,-,-]`, **front = 0+1 = 1**, and ***back = 3***.

If we call enqueue(5), we have `[-,(4,1,7,5),-,-,-]`, **front = 1**, and ***back = 3+1 = 4***.

5-3. Array Implementation Issues - Part 3

However, many dequeue and enqueue operations later, we may have `[-,-,-,-,-,(4,2,3]`, **front = 5**, and ***back*** = 7. By now, we cannot enqueue anything else albeit we have many empty spaces at the front of the array.

If we allow both **front** and **back** indices to "wrap back" to index 0 when they have reached index **M**-1, we effectively make the array "circular" and we can use the empty spaces.

For example, if we call enqueue(8) next, we have `[8),-,-,-,-,(4,2,3]`, **front = 5**, and ***back = (7+1)%8 = 0***.

5-4. Array Implementation Issues - Part 4

Yet, this does not solve the main problem of fixed-size array implementation. A few more enqueue operations later, we may have `[8,10,11,12,13],(6,2,3]`, **front = 5**, and ***back = 4***. At this point (**front = (back+1) % M**), we cannot enqueue anything else.

Do note that if we know that our queue size will never exceed the fixed array size **M**, then the circular array idea is actually already a good way to implement Queue ADT.

However, if we do not know the upper bound of queue size, we can enlarge (double) the size of the array, e.g., make **M = 2*8 = 16** (two-times larger), but that will entail re-copying the items from index **front** to ***back*** in a **slow** (but rare) O(**N**) process to have `[(6,2,3,8,10,11,12,13),,,,,,,,,,,,,,,,]`, **front = 0**, and ***back = 7***.

PS1: If you understand amortized analysis, this heavy O(**N**) cost when the circular array is full can actually be spread out so that each enqueue remains O(**1**) in amortized sense.

PS2: There is an alternative way to implement an efficient Queue using two Stacks. How?

5-5. Efficient Queue with Two Stacks

[This is a hidden slide]

5-6. Linked List to the Rescue

If we do not really know the the upper bound of queue size, then Singly Linked List (SLL) can be a good data structure to implement Queue ADT.

Recall that in a Queue, we only need the two extreme ends of the List, one for insertion (enqueue) only and one for removal (dequeue) only.

If we review this slide, we see that insertion **after tail** and removal **from head** in a Singly Linked List are fast, i.e., O(**1**). Thus, we designate the head/tail of the Singly Linked List as the front/back of the queue, respectively. Then, as the items in a Linked List are **not** stored contiguously in computer memory, our Linked List can grow and shrink as needed.

In our visualization, Queue is basically a **protected** Singly Linked List where we can only peek at the head item, enqueue a new item to one position after the current tail, e.g., try Enqueue(random-integer), and dequeue existing item from the head, e.g., try RemoveHead() (which is essentially a dequeue operation). All operations are O(**1**).

5-7. Queue Application

Queue ADT is usually used to simulate real queues.

One super important application of Queue ADT is inside the Breadth-First Search graph traversal algorithm.

6. Doubly Linked List (DLL)

Doubly Linked List (DLL) is 99% the same as its Singly Linked List version. The main difference is that now each vertex contains **two** pointers. The **next** pointer is the same as in Singly Linked List version in which it links item $\mathbf{a_i}$ with the next item $\mathbf{a_{i+1}}$, if exists. The additional **prev** pointer also links item $\mathbf{a_i}$ with the previous item $\mathbf{a_{i-1}}$, if exists.

The usage of **prev** pointers makes it possible to move/iterate **backwards** at the expense of two-times memory usage requirement as now each vertex records one additional pointer. The positive side effect of this ability to move backwards is now we can address the weak removal at tail case of the Singly Linked List.

In this visualization, notice that the edges in Doubly Linked List (and later Deque) are undirected (bidirectional) edges.

6-1. Remove(i) - At Tail (i = N-1), Revisited

The main problem of removal of the tail element in the Singly Linked List, even if we have direct access to the tail item via the tail pointer, is that we then have to update the tail pointer to point to one item just before the tail after such removal.

With Doubly Linked List ability to move **backwards**, we can find this item before the tail via `tail->prev`... Thus, we can implement removal of tail this way (in C++):

```
Vertex* tmp = tail; // remember tail item
tail = tail->prev; // the key step to achieve O(1) performance :O
tail->next = null; // remove this dangling reference
delete tmp; // remove the old tail
```

Now this operation is O(**1**). Try RemoveTail() on example DLL [22 (head)<->2<->77<->6<->43<->76<->89 (tail)].

6-2. Constant Factor Extra Step(s) Elsewhere

As we have one more pointer **prev** for each vertex, their values need to be updated too during each insertion or removal. Try all these operations on example DLL [22 (head)<->2<->77<->6<->43<->76<->89 (tail)].

Try InsertHead(50) — additional step: 22's **prev** pointer points to new head 50.

Try InsertTail(10) — additional step: 10's **prev** pointer points to old tail 89.

Try Insert(3, 44) — additional step: 6's/44's **prev** pointers point to 44/77, respectively.

Try RemoveHead() — set new head 2's **prev** pointer to null.

Try Remove(5) — set 89's **prev** pointer to 43.

7. Double-Ended Queue (Deque)

Double-ended queue (often abbreviated to deque, pronounced deck) is an Abstract Data Type that generalizes a Queue, for which elements can be added to or removed only from **either** the front (head) or back (tail).

In our visualization, Deque is basically a protected Doubly Linked List where we can only:
search the head/tail item (peek front/back),
insert a new item to the head/tail (try InsertHead(50) or InsertTail(10)), and
remove an existing item from the head/tail (try RemoveHead() or RemoveTail()).

All operations are O(**1**).

7-1. Deque Applications

Deque are used a few advanced applications, like finding the shortest paths 0/1-weighted graph using modified BFS, on some sliding window techniques, etc.

8. Summary

Create operation is the same for all five modes.

However there are minor differences for the search/insert/remove operations among the five modes.

For Stack, you can only peek/restricted-search, push/restricted-insert, and pop/restricted-remove from the top/head.

For Queue, you can only peek/restricted-search from the front (or sometimes, the back), push/restricted-insert from the back, and pop/restricted-remove from the front.

For Deque, you can peek/restricted-search, enqueue/restricted-insert, dequeue/restricted-remove from both front/back, but not from the middle.

Single (Singly) and Doubly Linked List do not have such restrictions.

9. Extras

We have reached the end of this e-Lecture.

But read ahead for a few extra challenges.

9-1. Potential Discussion Topics

The following are the more advanced insights about Linked List:

1. What happen if we don't store the tail pointer too?
2. What if we use dummy head?
3. What if the last tail item points back to the head item?
4. What need to be changed to allow duplicate items (a more general List ADT)?

9-2. Our Current Answers

[This is a hidden slide]

9-3. C++ STL and Java API Implementations

C++ STL:
forward_list (a Singly Linked List)
stack
queue
list (a Doubly Linked List)
deque (actually not using Doubly Linked List but another technique, see cppreference)

Java API:
LinkedList (already a Doubly Linked List)
Stack
Queue (actually an interface, usually implemented using LinkedList class)
Deque (actually an interface, usually implemented using LinkedList class)

9-4. Python and OCaml Standard Library

Python:
list for List/Stack/Queue
deque

OCaml:
List
Stack
Queue
No built-in support for Deque

9-5. Online Quiz

For a few more interesting questions about this data structure, please practice on Linked List training module.

9-6. Online Judge Exercises

We also have a few programming problems that somewhat requires the usage of this Linked List, Stack, Queue, or Deque data structure:
UVa 11988 - Broken Keyboard (a.k.a. Beiju Text),
Kattis - backspace, and
Kattis - integerlists.

Try them to consolidate and improve your understanding about this data structure. You are allowed to use C++ STL, Python standard library, or Java API if that simplifies your implementation.

Hash Table

1. Introduction

Hash Table is a data structure to map key to values (also called Table or Map Abstract Data Type/ADT). It uses a **hash function** to map large or even non-Integer keys into a small range of Integer indices (typically [0..hash_table_size-1]).

The probability of two distinct keys colliding into the same index is relatively high and each of this potential collision needs to be resolved to maintain data integrity.

There are several collision resolution strategies that will be highlighted in this visualization: Open Addressing (Linear Probing, Quadratic Probing, and Double Hashing) and Closed Addressing (Separate Chaining). Try clicking Search(7) for a sample animation of searching a specific value 7 in a randomly created Hash Table using Separate Chaining technique (duplicates are allowed).

2. Motivation

Hashing is an algorithm (via a hash function) that maps large data sets of variable length, called keys, not necessarily Integers, into smaller Integer data sets of a fixed length.

A Hash Table is a data structure that uses a hash function to efficiently map keys to values (Table or Map ADT), for efficient search/retrieval, insertion, and/or removals.

Hash Table is widely used in many kinds of computer software, particularly for associative arrays, database indexing, caches, and sets.

In this e-Lecture, we will digress to Table ADT, the basic ideas of Hashing, the discussion of Hash Functions before going into the details of Hash Table data structure itself.

2-1. Table ADT

A Table ADT must support **at least** the following three operations as efficient as possible:

1. Search(v) — determine if **v** exists in the ADT or not,
2. Insert(v) — insert **v** into the ADT,
3. Remove(v) — remove **v** from the ADT.

Hash Table is one possible good implementation for this Table ADT (the other one is this).

PS1: For two weaker implementations of Table ADT, you can click the respective link: unsorted array or a sorted array to read the detailed discussions.

PS2: In live class, you may want to compare the requirements of Table ADT vs List ADT.

2-2. Direct Addressing Table (DAT)

When the range of the **Integer** keys is **small**, e.g., [0..**M**-1], we can use an initially empty (Boolean) array **A** of size **M** and implement the following Table ADT operations **directly**:

1. Search(v): Check if **A[v]** is true (filled) or false (empty),
2. Insert(v): Set **A[v]** to be true (filled),
3. Remove(v): Set **A[v]** to be false (empty).

That's it, we use the small Integer key itself to determine the address in array **A**, hence the name **Direct Addressing**. It is clear that all three major Table ADT operations are O(**1**).

PS: This idea is also used elsewhere, e.g., in Counting Sort.

2-3. Example of DAT

In Singapore (as of Apr 2023), bus routes are numbered from [2..991].

Not all integers between [2..991] are currently used, e.g., there is no bus route 989 — Search(989) should return false. A new bus route **x** may be introduced, i.e., Insert(**x**) or an existing bus route **y** may be discontinued, i.e., Remove(**y**).

As the range of possible bus routes is **small**, to record the data whether a bus route number exists or not, we can use a DAT with a Boolean array of size 1000 (generally, it is useful to give a few extra buffer cells on top of the current largest bus number of 991).

2-4. Example of DAT with Satellite Data

Notice that we can always add **satellite data** instead of just using a Boolean array to record the existence of the keys.

For example, we can use an **associative** String array **A** instead to map a bus route number to its operator name, e.g.,

```
A[2] = "Go-Ahead Singapore",
A[10] = "SBS Transit",
A[183] = "Tower Transit Singapore",
A[188] = "SMRT Buses", etc.
```

Discussion: Can you think of a few other real-life DAT examples?

2-5. The Answer

[This is a hidden slide]

2-6. DAT Limitations

The keys must be (or can be easily mapped to) **non-negative Integer** values. Note that basic DAT has problem in the full version of the example in the previous few slides as there are actually variations of bus route numbers in Singapore, e.g., 96B, 151A, NR10, etc.

The range of keys must be **small**.
The memory usage will be (insanely) large if we have (insanely) large range.

The keys must be dense, i.e., not many gaps in the key values.
DAT will contain too many empty (and wasted) cells otherwise.

We will overcome these restrictions with hashing.

3. Hashing: Ideas

Using hashing, we can:

1. Map (some) **non-Integer** keys (e.g., Strings) to Integers keys,
2. Map **large** Integers to **smaller** Integers.

3-1. Phone Numbers Example

For example, we have **N** = 400 Singapore phone numbers (Singapore phone number has 8 digits, so there are up to 10^8 = 100M possible phone numbers in Singapore).

Instead of using a DAT and use a **gigantic** array up to size **M** = 100 Million, we can use the following simple hash function **h(v) = v%997**.

This way, we map 8 digits phone numbers **6675 2378** and **6874 4483** into up to 3 digits **h(6675 2378) = 237** and **h(6874 4483) = 336**, respectively. Therefore, we only need to prepare an array of size **M** = 997 (997 is a prime) instead of **M** = 100 Million.

3-2. Hash Table Preview

With hashing, we can now implement the following Table ADT operations using Integer array (instead of Boolean array) as follows:

1. Search(v): Check if **A[h(v)] != -1** (we use -1 for an empty cell assuming **v ≥ 0**),
2. Insert(v): Set **A[h(v)] = v** (we hash v into **h(v)** so we need to somehow record key **v**),
3. Remove(v): Set **A[h(v)] = -1** — to be elaborated further.

3-3. Hash Table with Satellite Data

If we have keys that map to satellite data and we want to record the original keys too, we can implement the Hash Table using pair of (Integer, satellite-data-type) array as follows:

1. Search(v): Return **A[h(v)]**, which is a **pair (v, satellite-data)**, possibly empty,
2. Insert(v, satellite-data): Set **A[h(v)] = pair(v, satellite-data)**,
3. Remove(v): Set **A[h(v)] = (empty pair)** — to be elaborated further.

However, by now you should notice that something is incomplete...

3-4. Collision

A hash function may, and quite likely, map **different keys (Integer or not)** into the **same Integer slot**, i.e., a **many-to-one** mapping instead of **one-to-one** mapping.

For example, **h(6675 2378) = 237** from three slides earlier and if we want to insert another phone number **6675 4372**, we will have a problem as **h(6675 4372) = 237** too.

This situation is called a **collision**, i.e., two (or more) keys have the **same hash value**.

3-5. Probability of Collision

The Birthday (von Mises) Paradox asks: 'How many people (number of keys) must be in a room (Hash Table) of size 365 seats (cells) before the probability that some person **share a birthday** (collision, two keys are hashed to the same cell), ignoring the leap years (i.e., all years have 365 days), becomes > 50 percent (i.e., more likely than not)?'

The answer, which maybe surprising for some of us, is Reveal.

Let's do some calculation.

3-6. The Calculation

Let **Q(n)** be the probability of **unique** birthday for **n** people in a room.
Q(n) = 365/365 × 364/365 × 363/365 × ... × (365-n+1)/365,

i.e., the first person's birthday can be any of the 365 days, the second person's birthday can be any of the 365 days except the first person's birthday, and so on.

Let **P(n)** be the probability of **same birthday** (collision) for **n** people in a room.
P(n) = 1-Q(n).

We compute that **P(23) = 0.507 > 0.5 (50%)**.

Thus, we only need **23 people** (a small amount of keys) in the room (Hash Table) of size 365 seats (cells) for a (more than) 50% chance collision to happen (the birthday of two different people in that room is one of 365 days/slots).

3-7. Two Important Issues

Issue 1: We have seen a simple hash function like the **h(v) = v%997** used in Phone Numbers example that maps large range of Integer keys into a smaller range of Integer keys, but how about non Integer keys? How to do such hashing efficiently?

Issue 2: We have seen that by hashing, or mapping, large range into smaller range, there will very likely be a collision. How to deal with them?

4. Hash Functions

How to create a good hash function with these desirable properties?

1. Fast to compute, i.e., in O(**1**),
2. Uses as minimum slots/Hash Table size **M** as possible,
3. Scatter the keys into different base addresses as uniformly as possible ∈ [0..**M**-1],
4. Experience as minimum collisions as possible.

4-1. Preliminaries

Suppose we have a hash table of size **M** where keys are used to identify the satellite-data and a specific hash function is used to compute a hash value.

A **hash value/hash code** of key **v** is computed from the key **v** with the use of a hash function to get an Integer in the range 0 to **M**-1. This hash value is used as the base/home index/address of the Hash Table entry for the satellite-data.

4-2. Example of a Bad Hash Function

Using the Phone Numbers example, if we we define **h(v) = floor(v/1 000 000)**,
i.e., we select the first two digits a phone number.

```
h(66 75 2378) = 66
h(68 74 4483) = 68
```

Discuss: What happen when you use that hash function? Hint: See this.

4-3. The Answer

[This is a hidden slide]

4-4. Perfect Hash Function

Before discussing the reality, let's discuss the ideal case: **perfect hash functions**.

A perfect hash function is a **one-to-one** mapping between keys and hash values, i.e., no collision at all. It is possible if all keys are known beforehand. For example, a compiler/interpreter search for reserved keywords. However, such cases are rare.

A minimal perfect hash function is achieved when the table size is the same as the number of keywords supplied. This case is even rarer.

If you are interested, you can explore GNU gperf, a freely available perfect hash function generator written in C++ that automatically constructs perfect functions (a C++ program) from a user supplied list of keywords.

4-5. Hashing Integer - Best Practice

People has tried various ways to hash a large range of Integers into a smaller range of Integers as uniformly as possible. In this e-Lecture, we jump directly to one of the best and most popular version: **h(v) = v%M**, i.e., map **v** into Hash Table of size **M** slots. The (%) is a modulo operator that gives the remainder after division. This is clearly fast, i.e., O(**1**) assuming that **v** does not exceed the natural Integer data type limit.

The Hash Table size **M** is set to be a reasonably large prime not near a power of 2, about 2+ times larger than the expected number of keys **N** that will ever be used in the Hash Table. This way, the load factor α = N/M < 0.5 — we shall see later that having low load factor, thereby sacrificing empty spaces, help improving Hash Table performance.

Discuss: What if we set **M** to be a power of 10 (decimal) or power of 2 (binary)?

4-6. The Answer

[This is a hidden slide]

4-7. Hashing String - Best Practice

People has also tried various ways to hash Strings into a small range of Integers as uniformly as possible. In this e-Lecture, we jump directly to one of the best and most popular version, shown below:

```
int hash_function(string v) { // assumption 1: v uses ['A'..'Z'] only
  int sum = 0;                // assumption 2: v is a short string
  for (auto& c : v) // for each character c in v
    sum = ((sum*26)%M + (c-'A'+1))%M; // M is table size
  return sum;
}
```

Interactive (M = ∞), i.e., the modulo operation has no effect
v = Enter string v, hash_string(v) = 0.

Discussion: In real life class, discuss the components of the hash function above, e.g., why loop through all characters?, will that be slower than O(**1**)?, why multiply with 26?, what if the string v uses more than just UPPERCASE chars?, etc.

4-8. The Answer

[This is a hidden slide]

5. Collision Resolution

There are two major ideas: **Closed Addressing** versus **Open Addressing** method.

In Closed Addressing, the Hash Table looks like an Adjacency List (a graph data structure). The hash code of a key gives its fixed/**closed** base address. Collision is resolved by appending the collided keys inside an auxiliary data structure (usually any form of List ADT) identified by the base address.

In Open Addressing, all hashed keys are located in a single array. The hash code of a key gives its base address. Collision is resolved by checking/probing multiple alternative addresses (hence the name **open**) in the table based on a certain rule.

5-1. Separate Chaining (SC)

Separate Chaining (SC) collision resolution technique is simple. We use **M** copies of auxiliary data structures, usually Doubly Linked Lists. If two keys **a** and **b** both have the same hash value **i**, both will be appended to the (front/back) of Doubly Linked List **i** (in this visualization, we append to the back in O(**1**) with help of tail pointer). That's it, where the keys will be slotted in is completely dependent on the hash function itself, hence we also call Separate Chaining as Closed Addressing collision resolution technique.

If we use Separate Chaining, the load factor α = **N**/**M** is the average length of the **M** lists (unlike in Open Addressing, α can be "slightly over 1.0") and it will determine the performance of Search(v) as we may have to explore α elements on average. As Remove(v) also requires Search(v), its performance is similar as Search(v). Insert(v) is clearly O(**1**).

If we can bound α to be a small constant (true if we know the expected largest **N** in our Hash Table application so that we can set up **M** accordingly), then all Search(v), Insert(v), and Remove(v) operations using Separate Chaining will be O(**1**).

5-2. Open Addressing (OA)

There are three Open Addressing (OA) collision resolution techniques discussed in this visualization: Linear Probing (LP), Quadratic Probing (QP), and Double Hashing (DH).

To switch between the three modes, please click on the respective header.

Let:
M = HT.length = the current hash table size,
base = (key%HT.length),
step = the current probing step,
secondary = smaller_prime - key%smaller_prime (to avoid zero — elaborated soon)

We will soon see that the probing sequences of the three modes are:
Linear Probing: i=(base+step*1) % M,
Quadratic Probing: i=(base+step*step) % M, and
Double Hashing: i=(base+step*secondary) % M.

All three OA techniques require that the load factor α = **N**/**M** < 1.0 (otherwise no more insertion is possible). If we can bound α to be a small constant (true if we know the expected largest **N** in our Hash Table application so that we can set up **M** accordingly, preferably < 0.5 for most OA variants), then all Search(v), Insert(v), and Remove(v) operations using Open Addressing will be O(**1**) — details omitted.

6. Visualization

View the visualization of Hash Table above.

In this visualization, we allow the insertion of duplicate keys (i.e., a *multi*set). Since a multiset is more general than a set, simply just insert distinct integers in this visualization if you want to see how Hash Table works on distict integer keys only.

Due to limited screen space, we switch from default (1.0x) scale to 0.5x scale whenever you want to visualize Hash Table size **M** ∈ [46..90] for OA techniques. The limit is a bit lower, i.e., **M** ∈ [20..31] for SC technique.

The Hash Table is visualized horizontally like an array where index 0 is placed at the leftmost of the first row and index **M**-1 is placed at the rightmost of the last row but the details are different when we are visualizing

Separate Chaining (only the top row) versus Open Addressing (usually spans multiple rows) collision resolution techniques.

6-1. Separate Chaining Version

For Separate Chaining (SC) collision resolution technique, the first row contains the **M** "H" (Head) pointers of **M** Doubly Linked Lists.

Then, each Doubly Linked List **i** contains all keys that are hashed into **i** in arbitrary order (in 0.5x scale, the vertex label is displayed on top of the smaller black dot). Mathematically, all keys that can be expressed as **i** (mod **M**) — including all duplicates of **i** — are hashed into DLL **i**. Again, we do not store any satellite data in this visualization.

6-2. Open Addressing Version

There are three Open Addressing collision resolution techniques discussed in this visualization: Linear Probing (LP), Quadratic Probing (QP), and Double Hashing (DH).

For all three techniques, each Hash Table cell is displayed as a vertex with cell value of [0..99] displayed as the vertex label (in 0.5x scale, the vertex label is displayed on top of the smaller black dot). Without loss of generality, we do not show any satellite data in this visualization as we concentrate only on the arrangement of the keys. We reserve value -1 to indicate an 'EMPTY cell' (visualized as a blank vertex) and -2 to indicate a 'DELETED cell' (visualized as a vertex with abbreviated label "DEL"). The cell indices ranging from [0..**M**-1] are shown as red label below each vertex (rows of 15 indices in 1.0x scale or rows of 25 indices in 0.5x scale).

7. Separate Chaining (SC)

Try Insert([9,16,23,30,37,44]) to see how Insert(v) operation works if we use Separate Chaining as collision resolution technique. On such random insertions, the performance is good and each insertion is clearly O(**1**).

However if we try Insert([68,90]), notice that all Integers {68,90} are 2 (modulo 11) so all of them will be appended into the (back of) Doubly Linked List 2. We will have a long chain in that list. Note that due to the screen limitation, we limit the length of each Doubly Linked List to be at maximum 6.

7-1. Search(35) and Remove(35)

Try Search(35) to see that Search(v) can be made to run in O(**1+α**).

Try Remove(35) to see that Remove(v) can be made to run in O(**1+α**) too.

If **α** is large, Separate Chaining performance is not really O(**1**). However, if we roughly know the potential maximum number of keys **N** that our application will ever use, then we can set table size **M** accordingly such that **α = N/M** is a very low positive (floating-point) number, thereby making Separate Chaining performances to be expected O(**1**).

8. Linear Probing (LP)

In **Linear Probing** collision resolution technique, we scan forwards one index at a time for the **next empty/deleted slot** (wrapping around when we have reached the last slot) whenever there is a collision.

For example, let's assume we start with an empty Hash Table **HT** with table size **M = HT.length** = 7 as shown above that uses index 0 to **M**-1 = 7-1 = 6. Notice that 7 is a prime number. The (primary) hash function is simple, **h(v) = v%M**.

This walk-through will show you the steps taken by Insert(v), Search(v), and Remove(v) operations when using Linear Probing as collision resolution technique.

8-1. Insert([18, 14, 21)

Now click Insert([18,14,21]) — three individual insertions in one command.

Recap (to be shown after you click the button above).

Formally, we describe Linear Probing index **i** as **i = (base+step*1) % M** where **base** is the (primary) hash value of key **v**, i.e., **h(v)** and **step** is the Linear Probing step starting from 1.

Tips: To do a quick mental calculation of a (small) Integer **V** modulo **M**, we simply subtract **V** with the largest multiple of **M** ≤ **V**, e.g., 18%7 = 18-14 = 4, as 14 is the largest multiple of 7 that is ≤ 18.

8-2. Insert([1, 35])

Now click Insert([1,35]) (on top of the first three values inserted in the previous slide).

Recap (to be shown after you click the button above)

8-3. Search(35) and Search(8)

Now we illustrate Search(v) operation while using Linear Probing as collision resolution technique. The steps taken are very similar as with Insert(v) operation, i.e., we start from the (primary) hash key value and check if we have found **v**, otherwise we move one index forward at a time (wrapping around if necessary) and recheck on whether we have found **v**. We stop when we encounter an empty cell which implies that **v** is not in Hash Table at all (as earlier Insert(v) operation would have placed **v** there otherwise).

Now click Search(35) — you should see probing sequence [0,1,2,3 (key 35 found)].

Now click Search(7) — [1,2,3,4, 5 (empty cell, so key 8 is not found in the Hash Table)].

8-4. Remove(v) - Preliminary

Now let's discuss Remove(v) operation.

If we just set **HT[i] = EMPTY** cell straightaway where **i** is the index that contains **v** (after linear probing if necessary), do you realize that we will cause a problem? Why?

Hint: Review the past three slides on how Insert(v) and Search(v) behave.

8-5. The Answer

[This is a hidden slide]

8-6. Remove(21)

Now let's see the complete Remove(v). If we find **v** at index **i** (after Linear Probing if necessary), we have to set **HT[i] = DELETED** (abbreviated as **DEL** in this visualization) where **DEL** is a special symbol (generally you should only use a symbol that is **not** used in your application) to indicate that cell can be by-passed if necessary by future Search(v), but can be overwritten by future Insert(w). This strategy is called **Lazy Deletion**.

Now click Remove(21) — [0,1 (key 21 found and we set **H[1] = DEL**)].

Afterwards, please continue the discussion in the next slide.

8-7. Search(35) Again

Now click Search(35) — [0,1 (bypassing that DELETED cell), 2,3 (found key 35)].

Imagine what would have happened if we *wrongly* set **H[1] = EMPTY**.

8-8. Insert(28) - Overwriting DEL

Now click Insert(28) — you should see probing sequence [0,1 (found a cell with DEL symbol)], so it is actually can be overwritten with a new value without affecting the correctness of future Search(v). Therefore, we put 28 in index 1.

8-9. Primary Clustering, Part 1

Although we can resolve collision with Linear Probing, it is not the most effective way.

We define a **cluster** to be a collection of consecutive occupied slots. A cluster that covers the base address of a key is called the **primary cluster** of the key.

Now notice that Linear Probing can create large primary clusters that will increase the running time of Search(v)/Insert(v)/Remove(v) operations beyond the advertised O(**1**).

See an example above with **M** = 31 and we have inserted 15 keys [0..14] so that they occupy cells [0..14] (α = 15/31 < 0.5). Now see how 'slow' Insert(31) (the 16th key) is.

8-10. Linear Probing Sequence

The probe sequence of Linear Probing can be formally described as follows:

```
 h(v) // base address
(h(v) + 1*1) % M // 1st probing step if there is a collision
(h(v) + 2*1) % M // 2nd probing step if there is still a collision
(h(v) + 3*1) % M // 3rd probing step if there is still a collision
...
(h(v) + k*1) % M // k-th probing step, etc...
```

During Insert(v), if there is a collision but there is an empty (or DEL) slot remains in the Hash Table, we are sure to find it after at most **M** Linear Probing steps, i.e., in O(**M**). And when we do, the collision will be resolved, but the primary cluster of the key **v** is expanded as a result and future Hash Table operations will get slower too. Try the slow Search(31) on the same Hash Table as in the previous slide but with many DEL markers (suppose {4, 5, 8, 9, 10, 12, 14} have just been deleted).

8-11. Primary Clustering, Part 2

In the previous slide (Primary Clustering, Part 1), we break the assumption that the hash function should uniformly distribute keys around [0..**M**-1]. In the next example, we will show that the problem of primary clustering can still happen even if the hash function distribute the keys into several relatively short primary clusters around [0..**M**-1].

On screen, you see **M** = 31 with 15 random integers between [0..99] inserted (there are several random but short primary clusters). If we then insert these next 4 keys {2, 9, 12, 1}, the first three keys will "plug" the three empty cells and accidentally annex (or combine) those neighboring (but previously disjointed) clusters into a (very) long primary cluster. So the next insertion of a key 1 that lands at (the beginning of) this long primary cluster will end up performing almost O(M) probing steps just to find an empty cell. Try Insert([2,9,12,1]).

9. Quadratic Probing (QP)

To reduce primary clustering, we can modify the probe sequence to:

```
 h(v) // base address
(h(v) + 1*1) % M // 1st probing step if there is a collision
(h(v) + 2*2) % M // 2nd probing step if there is still a collision
(h(v) + 3*3) % M // 3rd probing step if there is still a collision
...
(h(v) + k*k) % M // k-th probing step, etc...
```

That's it, the probe jumps quadratically, wrapping around the Hash Table as necessary.

A very common mistake as this is a different kind of Quadratic Probing:
Doing h(v), (h(v)+1) % M, (h(v)+1+4) % M, (h(v)+1+4+9) % M, ...

9-1. Insert(38)

Assume that we have called Insert(18) and Insert(10) into an initially empty Hash Table of size **M = HT.length** = 7. As 18%7 = 4 and 10%7 = 3, 18 and 3 do not collide and both reside in index 4 and 3 respectively as shown above.

Now, let's click Insert(38).

Recap (to be shown after you click the button above).

9-2. Remove(18) and Search(38) Again

Remove(x) and Search(y) operations are defined similarly. Just that this time we use Quadratic Probing instead of Linear Probing.

For example, assume that we have called Remove(18) after the previous slide and we mark **HT[4] = DEL**. If we then call Search(38), we will use the same Quadratic Probing sequence as with previous slide, but passing through **HT[4]** which marked as DELETED.

9-3. Better than Linear Probing?

In a glance, Quadratic Probing that jumps +1, +4, +9, +16, ... quadratically seems able to solve the primary clustering issue that we have with Linear Probing earlier, but is it the perfect collision resolution technique?

Try Insert([12,17]).

Do you realized what has just happened?

9-4. The Details

We can insert 12 easily as h(12) = 12%7 = 5 was empty previously (see above).

However we have major problem inserting key 17 even if we still have 3 empty slots as:
h(17) = 17%7 = 3 is already occupied by key 10,
(3+1*1) % 7 = 4 is already occupied by key 18,
(3+2*2) % 7 = 0 is already occupied by key 38,
(3+3*3) % 7 = 5 is already occupied by key 12,
(3+4*4) % 7 = 5 again is already occupied by key 12,
(3+5*5) % 7 = 0 again is already occupied by key 38,
(3+6*6) % 7 = 4 again is already occupied by key 18,
(3+7*7) % 7 = 3 again is already occupied by key 10,
it will **cycle forever** if we continue the Quadratic Probing...

Although we still have a few (3) empty cells, we are unable to insert this new value 17 into the Hash Table...

9-5. A Theorem

If α < 0.5 and **M** is a prime (> 3), then we can always find an empty slot using (this form of) Quadratic Probing. Recall: α is the load factor and **M** is the Hash Table size (HT.length).

If the two requirements above are satisfied, we can prove that the first **M**/2 Quadratic Probing indices, including the base address `h(v)` are all distinct and unique.

But there is no such guarantee beyond that. Hence if we want to use Quadratic Probing, we need to ensure that α < 0.5 (not enforced in this visualization but we do break the loop after **M** steps to prevent infinite loop).

9-6. A Proof

We will use proof by contradiction. We first assume that two Quadratic Probing steps:
x and y, x != y (let's say x < y), can yield the same address modulo **M**.

```
h(v) + x*x = h(v) + y*y (mod M)
x*x = y*y (mod M) // strike out h(v) from both sides
x*x - y*y = 0 (mod M) // move y*y to LHS
(x-y)*(x+y) = 0 (mod M) // rearrange the formula
```

Now, either `(x-y)` or `(x+y)` has to be equal to zero.
As our assumption says `x != y`, then `(x-y)` cannot be 0.
As `0 ≤ x < y ≤ (M/2)` and `M` is a prime > 3 (an odd Integer),
then `(x+y)` also cannot be 0 modulo `M`.

Contradiction!

So the first `M/2` Quadratic Probing steps cannot yield the same address modulo `M`
(if we set **M** to be a prime number greater than 3).

Discussion: Can we make Quadratic Probing able to use the other ~50% of the table cells?

9-7. Better Quadratic Probing

[This is a hidden slide]

9-8. Secondary Clustering

In Quadratic Probing, clusters are formed along the path of probing, instead of around the base address like in Linear Probing. These clusters are called **Secondary Clusters** and it is 'less visible' compared to the Primary Clusters that plagued the Linear Probing.

Secondary clusters are formed as a result of using the same pattern in probing by colliding keys, i.e., if two distinct keys have the same base address, their Quadratic Probing sequences are going to be the same.

To illustrate this, see the screen with **M** = 31. We have populated this Hash Table with only 10 keys (so load factor α = 10/31 ≤ 0.5) and the Hash Table looks 'sparse enough' (no visibly big primary cluster). However, if we then insert Insert(62,93), despite the fact that there are many (31-10 = 21) empty cells and 62 != 93 (different keys that ends up hashed into index 0), we end up doing 10 probing steps along this 'less visible' secondary cluster (notice that both {62, 93} follow similar Quadratic Probing sequences).

Secondary clustering in Quadratic Probing is not as bad as primary clustering in Linear Probing as a good hash function should theoretically disperse the keys into different base addresses ∈ [0..**M**-1] in the first place.

10. Double Hashing (DH)

To reduce primary and secondary clustering, we can modify the probe sequence to:

```
 h(v) // base address
(h(v) + 1*h2(v)) % M // 1st probing step if there is a collision
(h(v) + 2*h2(v)) % M // 2nd probing step if there is still a collision
(h(v) + 3*h2(v)) % M // 3rd probing step if there is still a collision
...
(h(v) + k*h2(v)) % M // k-th probing step, etc...
```

That's it, the probe jumps according to the value of the **second hash function** `h2(v)`, wrapping around the Hash Table as necessary.

10-1. Secondary Hash Function h2(v)

If `h2(v) = 1`, then Double Hashing works exactly the same as Linear Probing.
So we generally wants `h2(v) > 1` to avoid primary clustering.

If `h2(v) = 0`, then Double Hashing does not work for an obvious reason as any probing step multiplied by 0 remains 0, i.e. we stay at the base address forever during a collision. We need to avoid this.

Usually (for Integer keys), `h2(v) = M' - v%M'` where `M'` is a smaller prime than `M`.
This makes `h2(v)` ∈ [1..**M'**], which is diverse enough to avoid secondary clustering.

The usage of the secondary hash function makes it theoretically hard to have either primary or secondary clustering issue.

10-2. Insert([35, 42])

Click Insert([35,42]) to insert 35 and then 42 to the current Hash Table above.

Recap (to be shown after you click the button above).

10-3. Remove(17) and Search(35) Again

Remove(x) and Search(y) operations are defined similarly. Just that this time we use Double Hashing instead of Linear Probing or Quadratic Probing.

For example, assume that we have called Remove(17) after the previous slide and we mark **HT[3] = DEL**. If we then call Search(35), we will use the same Double Hashing sequence as with previous slide, but passing through **HT[3]** which marked as DELETED.

10-4. Good OA Collision Resolution Technique

In summary, a good Open Addressing collision resolution technique needs to:

1. Always find an empty slot if it exists,
2. Minimize clustering (of any kind),
3. Give different probe sequences when 2 different keys collide,
4. Fast, O(**1**).

Now, let's see the same test case that plagues Quadratic Probing earlier. Now try Insert(62,93) again. Although h(62) = h(93) = 0 and their collide with 31 that already occupy index 0, their probing steps are not the same: h2(62) = 29-62%29 = 25 is not the same as h2(93) = 29-93%29 = 23.

Discussion: Double Hashing seems to fit the bill. But... Is Double Hashing strategy flexible enough to be used as the default library implementation of a Hash Table? Or in a more general sense, which of the two collision resolution technique (Separate Chaining versus Open Addressing: Double Hashing) is the better one?

10-5. The (Current) Answer

[This is a hidden slide]

11. Extras

You have reached the end of the basic stuffs of this Hash Table data structure and we encourage you to explore further in the **Exploration Mode**.

However, we still have a few more interesting Hash Table challenges for you that are outlined in this section.

11-1. Rehash

The performance of Hash Table degrades when the load factor α gets higher. For (standard) Quadratic Probing collision resolution technique, insertions might fail when the Hash Table has α > 0.5.

If that happens, we can **rehash**. We build another Hash Table about twice as big with a new hash function. We go through all keys in the original Hash Table, recompute the new hash values, and re-insert the keys (with their satellite-data) into the new, bigger Hash Table, before finally we delete the older, smaller Hash Table.

A rule of thumb is to rehash when α ≥ 0.5 if using Open Addressing and when α > small constant (close to 1.0, as per requirement) if using Separate Chaining.

If we know the maximum number of total possible keys, we can always influence α to be a low number.

11-2. Hash Table Implementation

However, if you ever need to implement a Hash Table in C++, Python, or Java, and your keys are either Integers or Strings, you can use the built-in C++ STL, Python standard library, or Java API, respectively. They already have good built-in implementation of default hash functions for Integers or Strings.

See C++ STL std::unordered_map, std::unordered_set, Python dict, set, or Java HashMap, HashSet.

For C++, note that the std::multimap/std::multiset implementations are also exist where duplicate keys are allowed.

For OCaml, we can use Hashtbl.

However, here is our take of a simple Separate Chaining implementation: HashTableDemo.cpp | py | java.

11-3. Data Structure Combo?

[This is a hidden slide]

11-4. Alternative Data Structure for Table ADT

Hash Table is an extremely good data structure to implement Table ADT if the (Integer or String) keys only need to be mapped to satellite-data, with O(**1**) performance for Search(v), Insert(v), and Remove(v) operations if the Hash Table is set up properly.

However, if we need to do much more with the keys, we may need to use an alternative data structure.

11-5. Online Quiz

For a few more interesting questions about this data structure, please practice on Hash Table training module (no login is required, but short and of medium difficulty setting only).

However, for registered users, you should login and then go to the Main Training Page to officially clear this module and such achievement will be recorded in your user account.

11-6. Online Judge Exercises

Try to solve a few basic programming problems that somewhat requires the usage of Hash Table (especially if the input size is much larger):

1. Kattis - cd (the inputs are already sorted so alternative, non Hash Table solution exists; if the inputs are not sorted, this set intersection problem is best solved with help of a Hash Table),
2. Kattis - oddmanout (we can map large invitation codes into smaller range of integers; this is a practice of hashing (large range) of integers),
3. Kattis - whatdoesthefoxsay (we put sounds that are not fox into an unordered set; this is a practice of hashing strings).

Binary (Max) Heap

1. Introduction

A Binary (Max) Heap is a complete binary tree that maintains the Max Heap property.

Binary Heap is one possible data structure to model an efficient Priority Queue (PQ) Abstract Data Type (ADT). In a PQ, each element has a "priority" and an element with higher priority is served before an element with lower priority (ties are either simply resolved arbitrarily or broken with standard First-In First-Out (FIFO) rule as with a normal Queue). Try clicking ExtractMax() for a sample animation on extracting the max value of random Binary Heap above.

To focus the discussion scope, this visualization show a Binary **Max** Heap of integers where duplicates are allowed. See this for an easy conversion to Binary **Min** Heap. Generally, any other objects that can be compared can be stored in a Binary Max Heap, e.g., Binary Max Heap of floating points, etc.

1-1. Definitions

Complete Binary Tree: Every level in the binary tree, except possibly the last/lowest level, is completely filled, and all vertices in the last level are as far left as possible.

Binary Max Heap property: The parent of each vertex - except the root - contains value greater than (or equal to — we now allow duplicates) the value of that vertex. This is an easier-to-verify definition than the following alternative definition: The value of a vertex - except the leaf/leaves - must be greater than (or equal to) the value of its one (or two) child(ren).

1-2. Priority Queue ADT

Priority Queue (PQ) Abstract Data Type (ADT) is similar to normal Queue ADT, but with these two major operations:

1. Enqueue(**x**): Put a new element (key) **x** into the PQ (in some order),
2. **y** = Dequeue(): Return an existing element **y** that has the highest priority (key) in the PQ and if ties, return any.

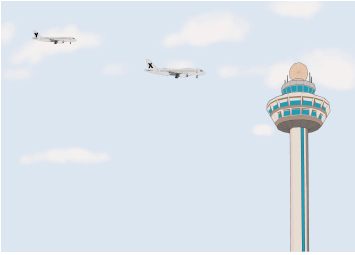
Discussion: Some PQ ADT reverts to First-In First-Out (FIFO) behavior of a normal Queue in the event there is a tie of highest priority (key) in the PQ. Does guaranteeing stability on equal highest priority (key) makes PQ ADT harder to implement?

1-3. Stability of Equal Highest Key

[This is a hidden slide]

1-4. Example

Imagine: You are an Air Traffic Controller (ATC) working in the control tower **T** of an airport. You have scheduled aircraft **X**/**Y** to land in the next 3/6 minutes, respectively. Both have enough fuel for at least the next 15 minutes and both are just 2 minutes away from your airport. You observe that your airport runway is clear at the moment.



In case you do not know, aircraft can be instructed to fly in holding pattern near the airport until the designated landing time.

1-5. For Live Lecture @ NUS Only

You have to attend the live lecture to figure out what happens next...

There will be two options presented to you and you will have to decide:

- Raise AND wave your hand if you choose option A,
- Raise your hand but do NOT wave it if you choose option B,

If none of the two options is reasonable for you, simply do nothing.

1-6. The Example - Continued

[This is a hidden slide]

1-7. PQ Examples

There are several potential usages of PQ ADT in real-life on top of what you have seen just now regarding ATC (only in live lecture).

Discussion: Can you mention a few other real-life situations where a PQ is needed?

1-8. Potential Answers

[This is a hidden slide]

1-9. Linear DS for PQ?

We are able to implement this PQ ADT using (circular) Array or Linked List but we will have slow (i.e., in O(**N**)) Enqueue or Dequeue operation.

Discussion: Why?

1-10. The Answer - Part 1

[This is a hidden slide]

1-11. The Answer - Part 2

[This is a hidden slide]

2. Visualisation + Max Heap Property

Now, let's view the visualisation of a (random) Binary (Max) Heap above. You should see a complete binary tree and all vertices except the root satisfy the Max Heap property (A[parent(i)] ≥ A[i]). Duplicate integer keys may appear (note that the stability of equal keys is not guaranteed).

You can Toggle the Visualization Mode between the visually more intuitive complete binary tree form or the compact array based implementation of a Binary (Max) Heap.

Quiz: **Based on this Binary (Max) Heap property, where will the largest integer be located?**

○ Can be anywhere
○ At one of the leaf
○ At the root

2-1. Binary Heap has O(log N) Height

Important fact to memorize at this point: If we have a Binary Heap of **N** elements, its height will not be taller than O(log **N**) since we will store it as a complete binary tree.

Simple analysis: The size **N** of a full (more than just a complete) binary tree of height **h** is always $\mathbf{N = 2^{(h+1)}-1}$, therefore $\mathbf{h = \log_2(N+1)-1 \sim= \log_2 N}$.

See example above with $\mathbf{N = 7 = 2^{(2+1)}-1}$ or $\mathbf{h = \log_2(7+1)-1 = 2}$.

This fact is important in the analysis of all Binary Heap-related operations.

2-2. 1-based Compact Array

A complete binary tree can be stored efficiently as a compact array A as there is no gap between vertices of a complete binary tree/elements of a compact array. To simplify the navigation operations below, we use 1-based array. VisuAlgo displays the index of each vertex as a red label below each vertex. Read those indices in sorted order from 1 to **N**, then you will see the vertices of the complete binary tree from top to down, left to right. To help you understand this, Toggle the Visualization Mode several times.

This way, we can implement basic binary tree traversal operations with simple index manipulations (with help of bit shift manipulation):

1. parent(i) = i>>1, index i divided by 2 (integer division),
2. left(i) = i<<1, index i multiplied by 2,
3. right(i) = (i<<1)+1, index i multiplied by 2 and added by 1.

Pro tip: Try opening two copies of VisuAlgo on two browser windows. Try to visualize the same Binary Max Heap in two different modes and compare them.

3. Binary (Max) Heap Operations

In this visualization, you can perform several Binary (Max) Heap operations:

1. **Create(A)** - O(**N** log **N**) version (**N** calls of **Insert(v)** below)
2. **Create(A)** - O(**N**) version
3. **Insert(v)** in O(log **N**) — you are allowed to insert duplicates
4. 3 versions of **ExtractMax()**:
   1. Once, in O(log **N**)
   2. **K** times, i.e., **PartialSort()**, in O(**K** log **N**), or
   3. **N** times, i.e., **HeapSort()**, in O(**N** log **N**)
5. **UpdateKey(i, newv)** in O(log **N** if **i** is known)
6. **Delete(i)** in O(log **N** if **i** is known)

There are a few other possible Binary (Max) Heap operations, but currently we do not elaborate them for pedagogical reason in a certain NUS module.

3-1. What Are The Extra Operations?

[This is a hidden slide]

4. Insert(v)

Insert(v): Insertion of a new item **v** into a Binary Max Heap can only be done at the *last index N plus 1* to maintain the compact array = complete binary tree property. However, the Max Heap property *may* still be violated. This operation then fixes Max Heap property from the insertion point **upwards** (if necessary) and stop when there is no more Max Heap property violation. Now try clicking Insert(v) several times to insert a few random **v** to the currently displayed Binary (Max Heap).

The fix Max Heap property upwards operation has no standard name. We call it **ShiftUp** but others may call it **BubbleUp** or **IncreaseKey** operation.

4-1. Why it is Correct?

Do you understand why starting from the insertion point (index **N**+1) upwards (at most until the root) and swapping a vertex with its parent when there is a Max Heap property violation during insertion is always a correct strategy?

4-2. The Answer

[This is a hidden slide]

4-3. Time Complexity Analysis

The time complexity of this **Insert(v)** operation is O(log **N**).

Discussion: Do you understand the derivation?

4-4. The Answer

[This is a hidden slide]

5. ExtractMax() - Once

ExtractMax(): The reporting and then the deletion of the maximum element (the root) of a Binary Max Heap requires an existing element to replace the root, otherwise the Binary Max Heap (a single complete binary tree, or 林/Lín in Chinese/tree) becomes two disjoint subtrees (two copies of 木/mù in Chinese/wood). That element must be the *last index* **N** for the same reason: To maintain the compact array = complete binary tree property.

Because we promote a leaf vertex to the root vertex of a Binary Max Heap, it will very likely violates the Max Heap property. ExtractMax() operation then fixes Binary Max Heap property from the root **downwards** by comparing the current value with the its child/the larger of its two children (if necessary). Now try ExtractMax() on the currently displayed Binary (Max) Heap.

The fix Max Heap property downwards operation has no standard name. We call it **ShiftDown** but others may call it **BubbleDown** or **Heapify** operation.

5-1. Why Compare with the Larger Child?

Why if a vertex has two children, we have to check (and possibly swap) that vertex with *the larger* of its two children during the downwards fix of Max Heap property?

Why can't we just compare with the left (or right, if exists) vertex only?

5-2. The Answer

[This is a hidden slide]

5-3. Time Complexity Analysis

The time complexity of this **ExtractMax()** operation is O(log **N**).

Discussion: Do you understand the derivation?

5-4. The Answer

[This is a hidden slide]

6. Binary Heap for Efficient PQ

Up to here, we have a data structure that can implement the two major operations of Priority Queue (PQ) ADT efficiently:

1. For **Enqueue(x)**, we can use **Insert(x)** in O(log **N**) time, and
2. For **y** = Dequeue(), we can use **y = ExtractMax()** in O(log **N**) time.

However, we can do a few more operations with Binary Heap.

7. Create(A) - Two Versions

Create(A): Creates a valid Binary (Max) Heap from an input array **A** of **N** integers (comma separated) into an initially empty Binary Max Heap.

There are two variants for this operations, one that is simpler but runs in O(**N** log **N**) and a more advanced technique that runs in O(**N**).

Pro tip: Try opening two copies of VisuAlgo on two browser windows. Execute different Create(A) versions on the worst case 'Sorted example' to see the *somewhat dramatic* differences of the two.

7-1. Create(A) - O(N log N)

Create(A) - O(N log N): Simply insert (that is, by calling **Insert(v)** operation) all **N** integers of the input array into an initially empty Binary Max Heap one by one.

Analysis: This operation is clearly O(**N** log **N**) as we call O(log **N**) **Insert(v)** operation **N** times. Let's examine the 'Sorted example' which is one of the hard case of this operation (Now try the Hard Case - O(N log N) where we show a case where **A = [1,2,3,4,5,6,7]** -- please be patient as this example will take some time to complete). If we insert values in increasing order into an initially empty Binary Max Heap, then every insertion triggers a path from the insertion point (a new leaf) upwards to the root.

7-2. Create(A) - O(N)

Create(A) - O(N): This faster version of **Create(A)** operation was invented by Robert W. Floyd in 1964. It takes advantage of the fact that a compact array = complete binary tree and all leaves (i.e., half of the vertices — see the next slide) are Binary Max Heap by default. This operation then fixes Binary Max Heap property (if necessary) only from the last internal vertex back to the root.

Analysis: A loose analysis gives another O(**N**/2 log **N**) = O(**N** log **N**) complexity but it is actually just O(2*N) = O(**N**) — details here. Now try the Hard Case - O(N) on the same input array **A = [1,2,3,4,5,6,7]** and see that on the same hard case as with the previous slide (but not the one that generates maximum number of swaps — try 'Diagonal Entry' test case), this operation is far superior than the O(**N** log **N**) version.

7-3. Many Leaf Vertices

Simple explanation on why half of Binary (Max) Heap of **N** (without loss of generality, let's assume that **N** is even) elements are leaves are as follows:

Suppose that the last leaf is at index **N**, then the parent of that last leaf is at index **i = N/2** (remember this slide). The left child of vertex **i+1**, if exists (it actually does not exist), will be **2*(i+1) = 2*(N/2+1) = N+2**, which exceeds index **N** (the last leaf) so index **i+1** must also be a leaf vertex that has no child. As Binary Heap indexing is consecutive, basically indices [**i+1 = N/2+1**, **i+2 = N/2+2**, ..., **N**], or half of the vertices, are leaves.

8. HeapSort()

HeapSort(): John William Joseph Williams invented **HeapSort()** algorithm in 1964, together with this Binary Heap data structure. **HeapSort()** operation (assuming the Binary Max Heap has been created in O(**N**)) is very easy. Simply call the O(log **N**) **ExtractMax()** operation **N** times. Now try HeapSort() on the currently displayed Binary (Max) Heap.

Simple Analysis: **HeapSort()** clearly runs in O(**N** log **N**) — an optimal comparison-based sorting algorithm.

Quiz: **In worst case scenario, HeapSort() is asymptotically faster than...**

☐ Insertion Sort
☐ Merge Sort
☐ Selection Sort
☐ Bubble Sort

8-1. Discussion

Although **HeapSort()** runs in θ(**N** log **N**) time for all (best/average/worst) cases, is it really the best *comparison-based* sorting algorithm?

Discussion: How about caching performance of **HeapSort()**?

8-2. The Answer

[This is a hidden slide]

9. Extras

You have reached the end of the basic stuffs of this Binary (Max) Heap data structure and we encourage you to explore further in the **Exploration Mode**.

However, we still have a few more interesting Binary (Max) Heap challenges for you that are outlined in this section.

When you have cleared them all, we invite you to study more advanced algorithms that use Priority Queue as (one of) its underlying data structure, like Prim's MST algorithm, Dijkstra's SSSP algorithm, A* search algorithm (not in VisuAlgo yet), a few other greedy-based algorithms, etc.

9-1. Create(A) - O(N) Analysis (1)

Earlier, we have seen that we can create Binary Max Heap from a random array of size **N** elements in O(**N**) instead of O(**N** log **N**). Now, we will properly analyze this tighter bound.

First, we need to recall that the height of a full binary tree of size **N** is $\log_2$ **N**.

Second, we need to realise that the cost to run `shiftDown(i)` operation is not the gross upper bound O(log **N**), but O(**h**) where **h** is the height of the subtree rooted at **i**.

Third, there are `ceil(N/2^(h+1))` vertices at height **h** in a full binary tree.

On the example full binary tree above with **N = 7** and **h = 2**, there are:
`ceil(7/2^(0+1))` = `4` vertices: {44,35,26,17} at height **h = 0**,
`ceil(7/2^(1+1))` = `2` vertices: {62,53} at height **h = 1**, and
`ceil(7/2^(2+1))` = `1` vertex: {71} at height **h = 2**.

9-2. Create(A) - O(N) Analysis (2)

Cost of Create(A), the O(**N**) version is thus:

$$\underbrace{\sum_{h=0}^{\lfloor \log(n) \rfloor}}_{\text{Sum over all levels}} \underbrace{\overbrace{\left\lceil \frac{n}{2^{h+1}} \right\rceil}^{\text{# of nodes at height h}} \overbrace{O(h)}^{\text{Cost to Heapify a node at height h}}}_{\text{Cost for a level}} = \sum_{h=0}^{\lfloor \log(n) \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil c * h = O\left(n \sum_{h=0}^{\lfloor \log(n) \rfloor} \frac{h}{2^h}\right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(2N) = O(N)$$

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2} = 2$$

$$x = 1/2$$

PS: If the formula is too complicated, a modern student can also use WolframAlpha instead.

9-3. PartialSort()

The faster O(**N**) Create Max Heap from a random array of **N** elements is important for getting a faster solution if we only need top **K** elements out of **N** items, i.e., **PartialSort()**.

After O(**N**) Create Max Heap, we can then call the O(log **N**) **ExtractMax()** operation **K** times to get the top **K** largest elements in the Binary (Max) Heap. Now try PartialSort() on the currently displayed Binary (Max) Heap.

Analysis: **PartialSort()** clearly runs in O(**N** + **K** log **N**) — an output-sensitive algorithm where the time complexity depends on the output size **K**. This is faster than the lower-bound of O(**N** log **N**) if we fully sort the entire **N** elements when **K** < **N**.

9-4. Easy Max to Min Heap Conversion

If we only deal with numbers (including this visualization that is restricted to integers only), it is easy to convert a Binary Max Heap into a Binary Min Heap without changing anything, or vice versa.

We can re-create a Binary Heap with the negation of every integer in the original Binary Heap. If we start with a Binary Max Heap, the resulting Binary Heap is a Binary Min Heap (if we ignore the negative symbols — see the picture above), and vice versa.

9-5. UpdateKey(i, newv)

For some Priority Queue applications (e.g., HeapDecreaseKey in Dijkstra's algorithm), we may have to modify (increase or decrease) the priority of an existing value that is already inserted into a Binary (Max) Heap. If the index **i** of the value is known, we can do the following simple strategy: Simply update **A[i] = newv** and then call **both shiftUp(i)** and **shiftDown(i)**. Only at most one of this Max Heap property restoration operation will be successful, i.e., **shiftUp(i)**/**shiftDown(i)** will be triggered if **newv** >/< old value of **A[parent(i)]**/**A[larger of the two children of i]**, respectively.

Thus, **UpdateKey(i, newv)** can be done in O(log **N**), provided we know index **i**.

9-6. Delete(i)

For some Priority Queue applications, we may have to delete an existing value that is already inserted into a Binary (Max) Heap (and this value happens to be not the root). Again, if the index **i** of the value is known, we can do the following simple strategy: Simply update **A[i] = A[1]+1** (a larger number greater than the current root), call **shiftUp(i)** (technically, **UpdateKey(i, A[1]+1)**). This will floats index **i** to be the new root, and from there, we can easily call **ExtractMax()** once to remove it.

Thus, **Delete(i)** can be done in O(log **N**), provided we know index **i**.

Discussion: Now for **UpdateKey(i, newv)** and **Delete(i)**, what if we are given **oldv** instead and thus we have to search for its location in the Binary (Max) Heap? Can we do this faster than O(**N**)?

9-7. The Answer

[This is a hidden slide]

9-8. Stability Issue

If there are duplicate keys, the standard implementation of Binary Heap as shown in this visualization does not guarantee stability. For example, if we insert three copies of {7, 7, 7}, e.g., {7a, 7b, and 7c} (suffix a, b, c are there only for clarity), in that order, into an initially empty Binary (Max) Heap. Then, upon first extraction, the root (7a) will be extracted first and the last existing leaf (7c) will replaces 7a. As 7c and 7b (without the suffixes) are equal (7 and 7), there is no swap happening and thus the second extract max will take out 7c instead of 7b first — not stable.

If we really need to guarantee stability of equal elements, we probably need to attach different suffixes as shown earlier to make those identical elements to be unique again.

9-9. Source Code

If you are looking for an implementation of Binary (Max) Heap to actually model a Priority Queue, then there is a good news.

C++ and Java already have built-in Priority Queue implementations that very likely use this data structure. They are C++ STL priority_queue (the default is a Max Priority Queue) and Java PriorityQueue (the default is a Min Priority Queue). However, the built-in implementation may not be suitable to do some PQ extended operations efficiently (details omitted for pedagogical reason in a certain NUS course).

Python heapq exists but its performance is rather slow. OCaml doesn't have built-in Priority Queue but we can use something else that is going to be mentioned in the other modules in VisuAlgo (the reason on why the details are omitted is the same as above).

PS: Heap Sort is likely used in C++ STL algorithm partial_sort.

Nevertheless, here is our implementation of BinaryHeapDemo.cpp | py | java.

9-10. Online Quiz

For a few more interesting questions about this data structure, please practice on Binary Heap training module (no login is required).

However, for NUS students, you should login using your official class account, officially clear this module, and such achievement will be recorded in your user account.

9-11. Online Judge Exercises

We also have a few programming problems that somewhat requires the usage of this Binary Heap data structure: UVa 01203 - Argus and Kattis - numbertree.

Try them to consolidate and improve your understanding about this data structure. You are allowed to use C++ STL priority_queue, Python heapq, or Java PriorityQueue if that simplifies your implementation.

9-12. Discussion

[This is a hidden slide]

9-13. Shocking Stuff

After spending one long lecture on Binary (Max) Heap, here is a jaw-dropping moment...

Binary (Max) Heap data structure is probably **not** the best data structure to implement (certain operations of) ADT Priority Queue...

Discussion: So what is the alternative data structure?

9-14. The Answer

[This is a hidden slide]